OnBuch+

SIGNED COURSE • PREMIUM

ICT • UPPER SIXTH | UPPER SIXTH — ICT

CHAPTER 9

DESCRIPTION OF PERIPHERAL DEVICE CONTROL MECHANISM

SUGGESTED TIME: 1 PERIODS

IN THIS CHAPTER

This chapter explains how the processor controls peripheral devices: the role of device controllers and drivers, and the three great control mechanisms — programmed I/O (polling), interrupt-driven I/O and Direct Memory Access (DMA). It equips the learner to compare these techniques and to relate them to the everyday behaviour of a computer system.

- By the end of this chapter I can define a peripheral device and classify peripherals as input, output, storage and communication devices.
- By the end of this chapter I can explain why the CPU cannot manage peripherals directly and needs control mechanisms.
- By the end of this chapter I can describe the role and structure of a device controller (interface adapter).
- By the end of this chapter I can explain the function of a device driver in the operating system.
- By the end of this chapter I can describe the programmed I/O (polling) mechanism, its steps and its limitations.
- By the end of this chapter I can describe the interrupt-driven I/O mechanism and the role of the Interrupt Service Routine (ISR).

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+

CONTENTS

1. Peripherals, Controllers and the Need for Control Mechanisms
2. Programmed I/O (Polling)
3. Interrupt-Driven I/O
4. Direct Memory Access (DMA) and Comparison of Mechanisms
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can define a peripheral device and classify peripherals as input, output, storage and communication devices.
- By the end of this chapter I can explain why the CPU cannot manage peripherals directly and needs control mechanisms.
- By the end of this chapter I can describe the role and structure of a device controller (interface adapter).
- By the end of this chapter I can explain the function of a device driver in the operating system.
- By the end of this chapter I can describe the programmed I/O (polling) mechanism, its steps and its limitations.
- By the end of this chapter I can describe the interrupt-driven I/O mechanism and the role of the Interrupt Service Routine (ISR).
- By the end of this chapter I can describe Direct Memory Access (DMA) and the role of the DMA controller.
- By the end of this chapter I can compare polling, interrupts and DMA in terms of CPU involvement, speed and suitable applications.
- By the end of this chapter I can identify the control mechanism used in common real-life situations (printing, disk transfer, keyboard input).

PREREQUISITES

- Structure and functioning of a computer: CPU, memory, buses (Lower Sixth)
- Von Neumann architecture and the fetch–decode–execute cycle
- Input/output units and examples of peripheral devices
- Basic role of the operating system as resource manager
- Notion of data transfer over address, data and control buses

1 PERIPHERALS, CONTROLLERS AND THE NEED FOR CONTROL MECHANISMS

At a busy cybercafé in Buea, the attendant prints an exam paper, copies files from a USB flash drive, and answers a WhatsApp call on the same computer — all at once. How does a single CPU manage the keyboard, printer, disk and network card without losing a single keystroke or mixing up the printout? Why does the computer sometimes freeze when a faulty flash drive is inserted? The answer lies in the mechanisms the processor uses to control peripheral devices. This chapter unveils how the CPU communicates with, supervises and synchronises all the devices connected to it.

1.1 What Is a Peripheral Device?

DEFINITION **Peripheral Device**

A **peripheral device** (or simply **peripheral**) is any hardware component connected to a computer system that expands its capabilities but is not part of the core architecture (CPU, main memory, motherboard chipset). Peripherals communicate with the CPU through a standardised interface and are typically located outside the main system unit, though some (like internal hard disks or graphics cards) reside inside the case.

Peripherals are classified by the direction of data flow relative to the computer:

Category	Description	Cameroonian Examples
Input	Send data to the computer.	Keyboard, mouse, scanner (scanning GCE registration forms), microphone (recording voice notes for WhatsApp), webcam.
Output	Receive data from the computer.	Monitor (viewing results on `www.gceboard.cm`), printer (printing admission letters), speakers, projector.
Input/Output (Storage)	Both send and receive data; retain data without power.	USB flash drive (carrying past questions), external hard disk, SSD, SD card (in a camera covering Mount Cameroon).
Communication	Exchange data with other systems/networks.	Modem (MTN/Orange 4G router), Network Interface Card (NIC), Bluetooth adapter, Wi-Fi dongle.

1.2 The Speed Mismatch Problem

The fundamental challenge in peripheral control is the enormous speed gap between the CPU and peripheral devices.

- **CPU speed:** Modern processors run at 2 GHz to 4 GHz — one clock cycle every 0.25 ns to 0.5 ns. Instructions execute in nanoseconds.
- **Peripheral speed:** Mechanical devices (printers, disks) operate in milliseconds (*e*-3 s). Even fast serial interfaces (USB 3.0, SATA) transfer data in microseconds (*e* - 6 s) per block.

$$\text{Speed ratio} \approx \frac{1\,\text{ms}}{1\,\text{ns}} = 10^6$$

The CPU is roughly **one million times faster** than a typical peripheral. If the CPU managed a device directly — waiting for a printer to finish a line before sending the next — it would spend 99.9999% of its time idle, a colossal waste of processing power.

DID YOU KNOW? **The Professor and the Photocopier**

Imagine a professor (CPU) who can read a page in one second. He sends a 100-page document to a slow photocopier (peripheral) that takes 10 seconds per page. If the professor stands by the machine waiting for each page to finish before feeding the next, he wastes 1000 seconds doing nothing. A smart professor gives the job to an assistant (controller) and returns to research while the copier works.

1.3 Diversity of Peripherals and the Need for Standardised Interfaces

Beyond speed, peripherals differ wildly in:

- **Data format:** Parallel (old printer port) vs. serial (USB, SATA); ASCII text vs. raw binary blocks.
- **Voltage levels:** TTL (5 V/3.3 V), RS-232 (±12 V), USB differential (±400 mV).
- **Protocols:** Handshaking signals (STROBE, ACK, BUSY), packet framing (USB), error correction (Wi-Fi).

The CPU cannot possibly speak every language. We need a **hardware intermediary** that presents a uniform, simple interface to the system bus (address, data, control lines) while handling the device-specific electrical and protocol details on the other side. That intermediary is the **device controller**.

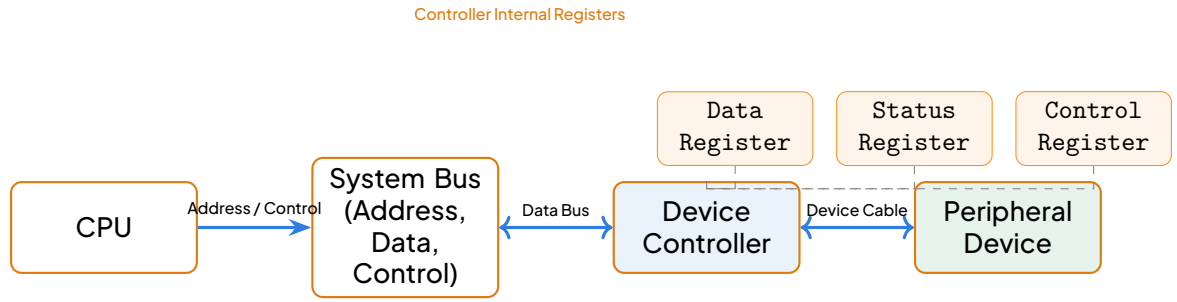
1.4 The Device Controller (Interface Adapter)

DEFINITION **Device Controller / Interface Adapter**

A **device controller** (also called an **interface adapter**) is an electronic circuit — a chip or a

small printed circuit board — that sits between the **system bus** and a **peripheral device**. Its role is to:

1. Translate bus signals (voltage, timing, width) into device signals.
2. Buffer data to absorb speed differences.
3. Report device state (ready, error) to the CPU.
4. Execute commands (read, write, reset) issued by the CPU.



Block diagram: CPU — System Bus — Device Controller (with its three registers) — Peripheral.

1.5 Internal Structure of a Controller

Every controller contains at least three registers, each mapped to a unique **I/O port address** (or memory address in memory-mapped I/O):

Data Register (DR) Holds the byte/word being transferred. **Read/Write** by CPU. Example: the ASCII code of a character to print.

Status Register (SR) Contains flags reflecting the device state. **Read-only** for CPU. Typical bits:

- Bit 0: `BUSY` (1 = device working, 0 = idle).
- Bit 1: `READY` / `DATA_VALID` (1 = data available in DR).
- Bit 2: `ERROR` (1 = paper jam, CRC error, etc.).

Control Register (CR) Receives commands from the CPU. **Write-only** for CPU. Typical bits:

- Bit 0: `START` / `READ_WRITE` (trigger transfer).
- Bit 1: `INT_ENABLE` (allow device to request interrupt).
- Bit 2: `RESET` (re-initialise device).

METHOD **Identifying the Role of Each Controller Register**

When analysing a controller description or exam question:

1. Data Register: Its content changes with the actual information being moved (characters, pixels, sectors). It is the only register whose value is meaningful to the application.
2. Status Register: You read it to ask a question ("Are you ready?", "Did an error occur?"). Its bits are flags, not data values.
3. Control Register: You write to it to give an order ("Start printing", "Enable interrupt"). Its bits are command triggers.

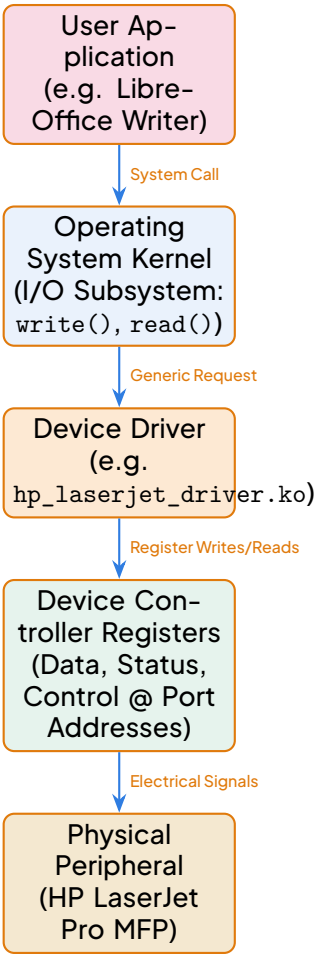
Mnemonic: Data = Delivery; Status = Spy (read only); Control = Command (write only).

1.6 The Device Driver: The Software Layer

Hardware alone is not enough. The operating system needs a standard way to ask "write this block" without knowing whether the target is an HP LaserJet, an Epson inkjet, or a network printer.

DEFINITION **Device Driver**

A **device driver** is a system software module (part of the OS kernel or loadable) that translates **generic, device-independent I/O requests** from the operating system into **device-specific command sequences** written into the controller's registers. It knows the exact register layout, command codes, timing requirements and error codes of one specific device model or family.



Layered software/hardware stack: Application → OS → Driver → Controller → Device.

EXEMPLE **Why Does Every Printer Need Its Own Driver?**

An HP LaserJet understands **PCL (Printer Command Language)**; an Epson understands **ESC/P**. The OS sends a generic `print_page(buffer)` request. The HP driver converts the buffer into PCL escape sequences, writes them byte-by-byte into the controller's Data Register, sets the Control Register `START` bit, and polls the Status Register `BUSY` bit. The Epson driver does the same but emits ESC/P codes. Without the correct driver, the printer receives gibberish.

WARNING **Driver vs. Controller — Classic Exam Trap**

- Device Controller = Hardware (silicon chip, PCB). Physically connected to the bus and the device cable. Cannot be downloaded.

- Device Driver = Software (file `.sys`, `.ko`, `.inf`). Stored on disk, loaded into RAM by the OS. Can be updated/downloaded.

Wrong: "I installed the controller from the CD." Right: "I installed the driver from the CD."

1.7 Overview of the Three Control Mechanisms

Now that we have the hardware (controller) and the low-level software (driver), how does the CPU actually orchestrate the transfer? Three mechanisms exist, differing in CPU involvement and efficiency:

Mechanism	CPU Role	Efficiency	Typical Use
1. Programmed I/O (Polling)	CPU actively loops, reading Status Register until device is ready.	Very low (CPU wasted).	Simple, slow devices; boot loaders; tiny microcontrollers.
2. Interrupt-Driven I/O	CPU does other work; device sends **interrupt request (IRQ)** when ready. CPU runs **ISR (Interrupt Service Routine)**.	High for slow/sporadic devices.	Keyboard, mouse, serial ports, network packets, printer "job done".
3. Direct Memory Access (DMA)	CPU sets up transfer (address, count, direction), then disengages. DMA controller moves data directly between device and RAM.	Highest for bulk transfers.	Disk I/O (SATA/NVMe), USB 3.0, sound cards, GPU frame buffers.

The rest of this chapter explores each mechanism in depth, with timing diagrams, interrupt vector tables and DMA controller registers.

KEY POINTS **Key Points of Section 1**

- A peripheral extends the computer; classified as Input, Output, Storage (I/O) or Communication.

- The speed mismatch (CPU ns vs device ms) makes direct CPU control impossible.
- The device controller (hardware) bridges the bus and the device; it contains Data, Status and Control registers mapped to I/O port addresses.
- The device driver (software) translates OS generic calls into controller-specific register operations.
- Three control mechanisms exist: Programmed I/O, Interrupt-Driven I/O, DMA — increasing in efficiency and hardware complexity.

WORKED EXAMPLE 1 **Identifying Controller Registers in a Scenario**

Statement: A temperature sensor controller has three 8-bit registers at port addresses `0x300`, `0x301`, `0x302`. The driver code reads:

```
while ((inb(0x301) & 0x01) == 0) ; // wait
char temp = inb(0x300);
outb(0x302, 0x04); // start next conversion
```

Identify which port is the Data, Status and Control Register. Justify.

SOLUTION

Detailed Solution:

1. `inb(0x301) & 0x01` reads a register and tests bit 0 in a loop ("wait while bit 0 is 0"). This is polling a status flag (e.g., `READY` or `BUSY`). ⇒ Port 0×301 = Status Register (Read-only).
2. `inb(0x300)` reads a value into variable `temp`. This value is the measured temperature — the actual data. ⇒ Port 0×300 = Data Register (Read-only for input device).
3. `outb(0x302, 0x04)` writes the constant `0x04` (binary `00000100`) to a register. This commands the device (bit 2 set = `START_CONVERSION`). ⇒ Port 0×302 = Control Register (Write-only).

Answer: 0×300 = Data Register, 0×301 = Status Register, 0×302 = Control Register.

2 PROGRAMMED I/O (POLLING)

In the previous section we saw that every peripheral device is attached to the system through a **controller** (or device interface) which exposes a small set of **registers** to the CPU: a **status register** (readable, tells whether the device is ready, busy or in error), a **data register** (where the

data to be transferred is placed or read), and a **command register** (writable, used to start an operation). We also saw that the fundamental question of input/output management is: who does the work of the transfer, and how does the CPU know when the device is ready? The simplest possible answer to that question is the subject of this section: let the CPU do everything, and let it ask the device, again and again, whether it is ready. This mechanism is called **programmed I/O**, and its waiting technique is called **polling** or **busy-waiting**.

2.1 Intuition: the shopkeeper who keeps checking the door

Imagine a photocopy shop in Buea. The shopkeeper must deliver a document to a customer who said: "I will knock when I arrive." Two strategies are possible. In the first, the shopkeeper stops all other work, stands at the door, opens it every few seconds and asks "Are you there?" until the customer finally arrives. In the second, the shopkeeper continues serving other customers and simply reacts when the knock comes. The first strategy is **polling**: simple, requiring no bell or doorbell (no extra hardware), but the shopkeeper's time is entirely wasted while waiting. The second strategy is the interrupt-driven approach we shall study in the next section.

A computer behaves exactly like the first shopkeeper when it uses programmed I/O: the CPU executes a small loop that reads the device's status register over and over — "ready? ready? ready?" — until the device finally answers "yes". During all that time, the CPU can do nothing else.

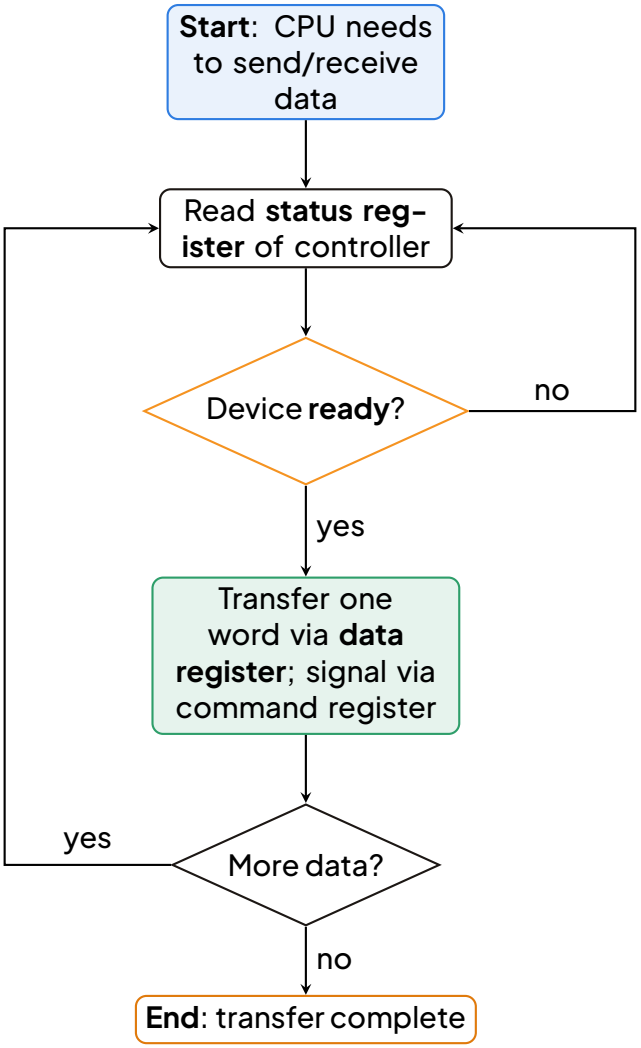
DEFINITION **Programmed I/O, polling, busy-waiting**

Programmed I/O is a peripheral control mechanism in which the CPU executes the entire data transfer itself, word by word, by explicitly reading and writing the registers of the device controller, under the control of an I/O program.
Polling (also called **busy-waiting**) is the waiting technique used in programmed I/O: the CPU repeatedly reads the status register of the controller in a loop until the "ready" flag indicates that the device can accept (or supply) the next unit of data. While polling, the CPU is busy doing nothing useful — hence the name busy-waiting.

2.2 The polling loop: principle and flowchart

The heart of programmed I/O is a very short loop with three logical steps:

1. **Read** the status register of the controller.
2. **Test** the ready bit. If the device is not ready, go back to step 1 (this is the waiting loop).
3. When the device is ready, **transfer** one unit of data (one byte or one word) by writing to (or reading from) the data register, then possibly write to the command register to signal the transfer. If more data remains, go back to step 1 for the next unit.



Flowchart of the polling loop in programmed I/O. The “no” branch from the ready test is the busy-waiting loop: the CPU consumes time without doing useful work.

PROPERTY **Total CPU involvement**

In programmed I/O, the CPU is involved in 100 % of the transfer: it executes one instruction (or more) for every status check and for every word transferred. No data ever moves between memory and the device without passing through the CPU, and no waiting happens without consuming CPU cycles. (This property is directly examinable: it is the basis of the comparison with interrupt-driven I/O and DMA.)

2.3 Worked scenario: printing a character by polling

Let us follow, step by step, what happens when a program prints the single character `'A'` on a

printer using programmed I/O. Suppose the printer controller has a status register whose bit 0 is the ready flag (1 = printer can accept a character, 0 = busy).

METHOD **Describing a programmed I/O transfer step by step**

To describe any programmed I/O transfer in an examination, always follow these numbered steps:

1. Initialise: the CPU writes a command into the command register if the device must be prepared (e.g. select the printer).
2. Poll: the CPU reads the status register and tests the ready bit.
3. Wait if busy: if the device is not ready, repeat step 2 (busy-waiting loop).
4. Transfer: when ready, the CPU writes one byte/word into the data register (output) or reads one byte/word from it (input).
5. Signal: the CPU writes to the command register to tell the controller to act on the data (e.g. “print the character”).
6. Repeat: if more data remains, return to step 2 for the next unit; otherwise the transfer is complete.

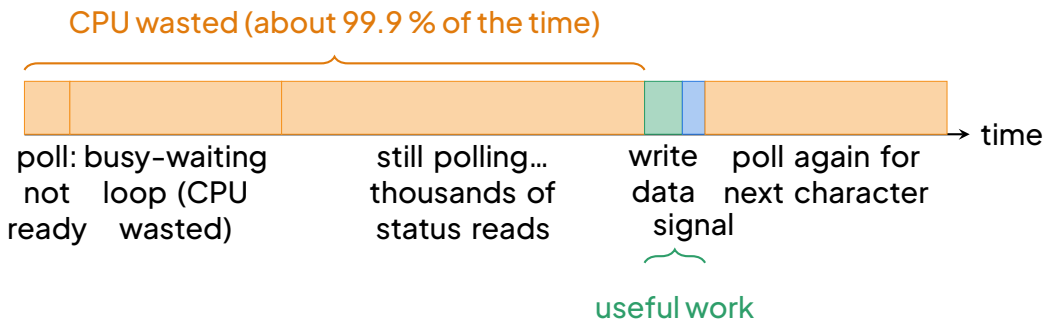
EXEMPLE **Printing `'A'` by polling**

1. The CPU reads the printer status register: ready bit = 0 (the printer is still finishing the previous line). The CPU loops: read, test, read, test… perhaps millions of times.
2. After some milliseconds, the ready bit becomes 1.
3. The CPU writes the code of `'A'` (65 in ASCII) into the data register.
4. The CPU writes “print” into the command register; the controller clears the ready bit and starts printing.
5. To print the next character, the CPU returns to step 1.

A full page of about 3 000 characters therefore requires 3 000 polling loops, each possibly containing thousands of useless status reads.

2.4 Where the CPU time goes: a timeline

The inefficiency of polling becomes striking when we compare the speed of the CPU with the speed of a mechanical device. A modern CPU executes billions of instructions per second, while a printer may print only a few hundred characters per second. Almost all the transfer time is therefore spent waiting.



Timeline of one character printed by polling. The useful work (writing the data and signalling) is a tiny fraction of the total time; the rest is busy-waiting.

WORKED EXAMPLE 2 **CPU time wasted by polling**

A CPU executes 10^9 instructions per second. A printer accepts one character every 5 ms. Printing one character by polling requires: one status read + test every loop iteration (2 instructions per iteration), and 10 useful instructions to write the data and signal once the device is ready. Estimate the number of instructions executed to print one character, and the percentage that is wasted.

SOLUTION

Solution. During 5 ms = 5×10^{-3} s, the CPU can execute

$$N = 10^9 \times 5 \times 10^{-3} = 5 \times 10^6 \text{ instructions.}$$

Almost all of these are polling iterations: about $\frac{5 \times 10^6}{2} = 2.5 \times 10^6$ loop iterations of 2 instructions each, plus the 10 useful instructions at the end. The wasted fraction is

$$\frac{5 \times 10^6 - 10}{5 \times 10^6} \approx 99.9998\,\%.$$

Conclusion: virtually all the CPU time is consumed by the waiting loop. With interrupt-driven I/O or DMA, these 5 million instruction slots could have been used to run other programs.

2.5 Advantages and disadvantages of programmed I/O

Aspect	Explanation
Advantage: simplicity	The I/O program is a short loop; easy to write, easy to understand, easy to debug.
Advantage: no extra hardware	No interrupt controller, no DMA controller is needed; the cheapest possible design.
Advantage: predictability	The CPU controls every instant of the transfer; timing is fully deterministic, which matters in some real-time systems.
Disadvantage: wasted CPU time	The busy-waiting loop consumes CPU cycles that could serve other programs; catastrophic with slow devices (printers, keyboards, disks).
Disadvantage: poor multitasking	While polling one device, the CPU cannot run other processes; the whole machine appears to "freeze".

DID YOU KNOW? **The "frozen" computer**

When an old computer seems to freeze completely while printing, or when a simple program stops responding while reading from a slow device, you are very often observing the symptom of busy-waiting: the CPU is trapped in a polling loop and cannot even update the screen or react to the mouse. Modern operating systems avoid this for slow devices by using interrupts and DMA — which is exactly the subject of the next sections.

2.6 When is polling acceptable?

Polling is not a "bad" mechanism; it is simply a mechanism whose cost must match the situation. It remains the right choice in several typical cases:

- **Very fast devices**: if the device is almost always ready (e.g. a high-speed device register on the same board), the loop exits immediately and polling is cheaper than managing interrupts.
- **Simple embedded systems**: a microwave oven controller or a simple sensor reader has only one task; there is nothing else for the CPU to do, so wasting its time costs nothing.
- **Critical real-time loops**: when a reaction must occur within a guaranteed, extremely short delay (e.g. an emergency stop on a machine), polling gives a deterministic response, whereas interrupts introduce small, variable delays.
- **Boot code and diagnostics**: before an operating system is loaded, firmware often uses polling because interrupt mechanisms are not yet configured.

WARNING **Common mistake: "polling is wrong"**

Do not write in an examination that polling is an "incorrect" or "useless" mechanism. Polling is correct — the data is transferred reliably — it is simply inefficient for slow de-

vices and multitasking environments. For fast devices, dedicated embedded systems and hard real-time constraints, polling is often the best choice. The examinable skill is to justify the choice of mechanism according to the device speed and the system context, not to condemn polling absolutely.

EXERCISE 1 **Analysing a polling situation**

A point-of-sale terminal in a Douala supermarket reads barcodes with a scanner. The scanner controller has a status register (bit 0 = "a code has been scanned") and a data register.

1. Describe, in numbered steps, how the terminal reads one barcode using programmed I/O.
2. Give one reason why polling is acceptable here, and one situation in which it would become a problem.

SOLUTION **Exercise 1**

1. Steps of the transfer:

1. The CPU reads the scanner status register.
2. If bit 0 is 0 (no code scanned), the CPU loops back to step 1 (busy-waiting).
3. When bit 0 is 1, the CPU reads the barcode value from the data register.
4. The CPU writes to the command register to acknowledge and reset the scanner.
5. The CPU returns to step 1 to wait for the next product.

2. Polling is acceptable because the terminal has essentially one task (scanning products), so the waiting time is not wasted on other work. It becomes a problem if the same terminal must simultaneously process mobile money payments, update stock over the network and run a display: while trapped in the polling loop, the CPU would delay all these other activities, and an interrupt-driven design would then be preferable.

KEY POINTS **Key points on programmed I/O**

- In **programmed I/O**, the CPU performs the whole transfer itself, word by word, through the controller's status, data and command registers.
- **Polling** (busy-waiting) is the loop in which the CPU repeatedly reads the status regis-

ter until the device is ready.

- The CPU is involved in 100 % of the transfer; with slow devices, the overwhelming majority of its time is wasted in the waiting loop.
- Advantages: simplicity, no extra hardware, deterministic timing. Disadvantages: wasted CPU time, incompatible with efficient multitasking.
- Polling remains appropriate for very fast devices, simple embedded systems and critical real-time loops — it is inefficient, not "wrong".

3 INTERRUPT-DRIVEN I/O

In the previous section we studied **programmed I/O** (polling): the CPU repeatedly reads the status register of a device controller in a loop, asking "Are you ready?" until the device answers "Yes". This mechanism is simple and reliable, but it has a fundamental weakness: while a slow device (a keyboard, a printer, a disk) prepares its data, the CPU spends all its time asking the same useless question. A processor capable of executing billions of instructions per second ends up waiting for a human typist who presses perhaps five keys per second. This is an enormous waste of processing power.

The natural question is: why should the CPU keep asking? In everyday life, you do not stand in front of your phone waiting for a WhatsApp message; you continue your work, and the phone rings when the message arrives. Interrupt-driven I/O applies exactly this idea to the machine: instead of the CPU polling the device, the device notifies the CPU when it needs attention. This section defines the interrupt, describes the hardware that carries it, traces the complete treatment of an interrupt step by step, and evaluates the strengths and limits of the mechanism.

3.1 The Idea of an Interrupt

DEFINITION **Interrupt**

An **interrupt** is a hardware signal sent by a device controller to the CPU to request its attention. When the CPU accepts the interrupt, it suspends the program it is currently executing, executes a special routine that services the device, and then resumes the suspended program exactly where it left off.

The signal travels on a dedicated wire called the **IRQ** line (Interrupt ReQuest). Each device controller is connected to an IRQ line. Because a computer has many devices (keyboard, mouse, timer, disk, network card) but the CPU has very few interrupt pins, the IRQ lines are not connected directly to the processor: they pass through an **interrupt controller**, historically the **PIC** (Programmable Interrupt Controller, the Intel 8259) and today the **APIC** (Advanced Programmable Interrupt Controller) found in modern multi-core machines.

The interrupt controller plays the role of an intelligent receptionist:

- it **collects** requests from all the devices;
- it **arbitrates** between simultaneous requests according to **priorities** (for example, the timer or a disk error is more urgent than a keyboard key);
- it **identifies** the requesting device to the CPU by supplying an interrupt number;
- it can **mask** (block) selected interrupts when the operating system decides they must wait.

DEFINITION **Interrupt Service Routine (ISR)**

An **Interrupt Service Routine** (**ISR**), also called an interrupt handler, is a small program, part of the operating system, whose job is to service one particular interrupt: read the data from the device controller, acknowledge the interrupt, and store the data where the system expects it.

3.2 How the CPU Treats an Interrupt: Context Saving

The CPU cannot simply abandon the running program: that program is using registers, the accumulator, the program counter, and status flags. If the ISR modified them freely, the suspended program would resume with corrupted values and produce wrong results. The treatment of an interrupt therefore follows a strict protocol, built around the idea of the **context**: the set of register values (program counter, status register, general registers) that completely describes the state of the running program at the instant of the interruption.

METHOD **Tracing the treatment of an interrupt**

To describe, step by step, how an interrupt is handled from the device signal to the resumption of the program:

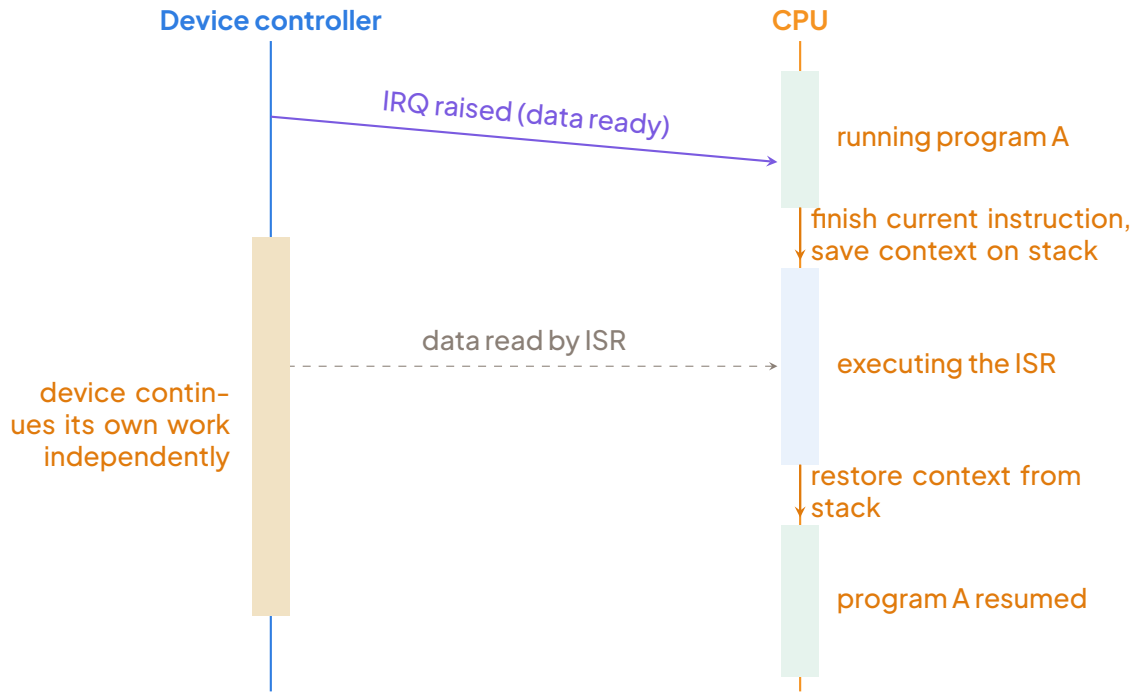
1. Request. The device controller raises its IRQ line; the interrupt controller forwards the request, with the interrupt number, to the CPU.
2. Completion of the current instruction. The CPU does not stop in the middle of an instruction: it finishes the instruction in progress, then checks the interrupt signal.
3. Masking check. If that interrupt is masked (disabled), the CPU ignores it for the moment; otherwise it accepts it.
4. Context saving. The CPU saves the context of the running program — at least the program counter and the status register, usually also the general registers — on the stack.
5. Locating the ISR. Using the interrupt number, the CPU looks up the address of the corresponding ISR in the interrupt vector table and jumps to it.
6. Servicing. The ISR executes: it reads or writes the device data and acknowledges the

interrupt at the controller so that the request line is released.

7. Context restoring. The saved registers are popped back from the stack.
8. Resumption. The CPU continues the suspended program from the exact instruction where it was interrupted. The program "notices nothing".

WARNING **The CPU never stops working**

A very common mistake is to write that "when an interrupt occurs, the CPU stops working". This is false. The CPU suspends the current program, services the device by executing the ISR (it is still working!), and then resumes the program. An interrupt is a change of task, not a shutdown. Two other frequent confusions: the CPU finishes its current instruction before reacting — it does not abandon an instruction halfway; and the interrupt is generated by the device controller, not by the CPU itself.



Sequence of an interrupt-driven transfer: the device raises the IRQ, the CPU finishes its current instruction, saves the context, executes the ISR, then resumes the interrupted program.

3.3 Worked Scenario: the Keyboard

EXEMPLE **A key is pressed**

Imagine a student typing an essay in a word processor. Between two keystrokes, the CPU is busy reformatting the paragraph, checking spelling, updating the display — useful work. Here is what happens when the student presses the letter "A":

1. The keyboard controller detects the key press, stores the scan code in its data register, and raises its IRQ line (historically IRQ1).
2. The interrupt controller forwards the request to the CPU with the keyboard's interrupt number.
3. The CPU finishes its current instruction, then saves the context of the word processor on the stack.
4. It consults the interrupt vector table, finds the address of the keyboard ISR, and jumps to it.
5. The keyboard ISR reads the scan code from the controller's data register, translates it, places the character "A" into the keyboard buffer of the operating system, and acknowledges the interrupt.
6. The context is restored; the CPU resumes the word processor, which will find "A" in the buffer and display it.

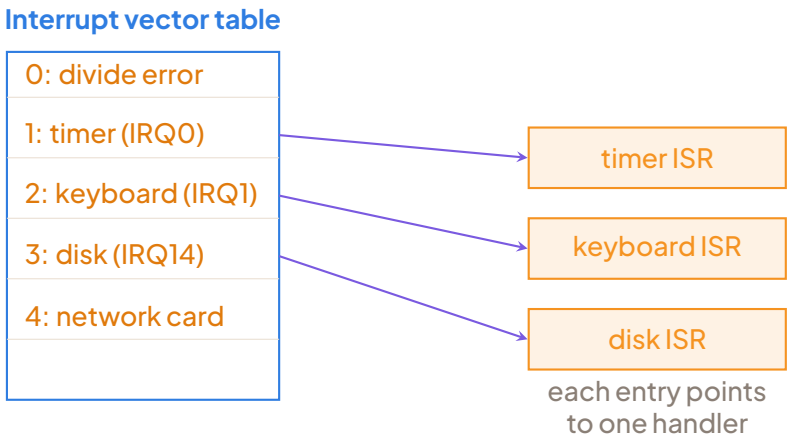
The whole service takes a few microseconds. If the student types five characters per second, the CPU is disturbed for a negligible fraction of its time — compare this with polling, where it would have asked "any key?" millions of times per second for nothing.

3.4 The Interrupt Vector Table, Masking and Priorities

Each device raises a different interrupt, and each interrupt needs its own ISR. How does the CPU know where the correct ISR is located in memory? It uses a data structure set up by the operating system at boot time.

DEFINITION **Interrupt Vector Table**

The **interrupt vector table** is a table in memory, initialised by the operating system, in which entry number n contains the address of the ISR associated with interrupt number n. When interrupt n occurs, the CPU uses n as an index into the table, reads the address stored there, and jumps to it.



The interrupt vector table: the interrupt number indexes the table, and each entry holds the address of the corresponding ISR (timer, keyboard, disk, …).

Two refinements make the mechanism usable in a real system:

Masking. Each interrupt source can be masked (disabled) by setting a bit in a **mask register**, either in the CPU or in the interrupt controller. A masked interrupt is not lost forever: it remains pending and is treated when unmasked. This is essential when the operating system executes a critical section of code that must not be disturbed. A few interrupts, called non-maskable interrupts (NMI), such as a memory hardware failure, can never be masked.

Priorities. When several devices request attention at the same time, the interrupt controller serves them in order of **priority**: a high-priority interrupt (timer, disk error) is treated before a low-priority one (keyboard). In many systems, a high-priority interrupt can even interrupt the ISR of a low-priority one — this is called nested interrupts.

3.5 Advantages and Limitations

PROPERTY **What interrupt-driven I/O achieves — and what it does not**

Interrupt-driven I/O frees the CPU during the device operation: while the device prepares its data, the CPU executes other programs, and it is only called upon when the device is ready. However, the CPU remains involved in each transfer: for every unit of data (every key, every byte or word), an interrupt occurs and the CPU must execute the ISR to move the data itself. (This property is qualitative; no proof is examinable, but you must be able to justify both halves with examples.)

Advantages	Disadvantages
No busy waiting: the CPU does useful work while the device is busy.	**Context-switching overhead**: saving and restoring registers costs time on every interrupt.
The device is serviced quickly after becoming ready (good responsiveness).	The CPU still performs every data transfer itself, instruction by instruction.
Several devices coexist thanks to priorities and masking.	With very fast devices, interrupts arrive so often that the CPU spends all its time entering and leaving ISRs: this is an interrupt storm (or live-lock).

The last disadvantage is crucial. For a keyboard (a few events per second), interrupts are perfect. For a gigabit network card receiving hundreds of thousands of small packets per second, the overhead of one context switch per packet becomes unbearable: the CPU does nothing but save and restore registers. This is precisely the limitation that motivates the next mechanism of the chapter, **Direct Memory Access** (DMA), in which the controller transfers whole blocks of data to memory without the CPU and interrupts it only once, at the end of the block.

DID YOU KNOW? **Software interrupts and exceptions (exam extra)**

Not all interrupts come from hardware. A software interrupt is triggered deliberately by an instruction (for example a system call, when a program asks the operating system to read a file); an exception is triggered automatically by an abnormal event detected during execution (division by zero, page fault, invalid instruction). The three categories share the same machinery — vector table, context saving, handler — but their causes differ. In the GCE, if you are asked to define an interrupt in this chapter, speak of the hardware interrupt sent by a device controller, and mention the other two only to show you can distinguish them.

WORKED EXAMPLE 3 **Polling or interrupts?**

A point-of-sale terminal in a Douala supermarket reads barcodes with a scanner (a few items per minute) and drives a receipt printer. The designer hesitates between programmed I/O and interrupt-driven I/O for the scanner. Advise him, with two arguments.

SOLUTION

Advice: interrupt-driven I/O.

- Argument 1 (CPU utilisation). Barcodes arrive rarely and unpredictably. With polling, the CPU would waste most of its time reading the scanner's status register and finding "not ready". With interrupts, the CPU serves customers' computations (totals, stock updates) and is only disturbed when a barcode actually arrives.
- Argument 2 (responsiveness without waste). When the cashier scans an item, the scanner controller raises an IRQ; the CPU suspends its task, the ISR reads the code,

and the program resumes. The item is registered immediately, with no polling loop and no wasted cycles.

Note that the volume of data is tiny (one code per scan), so the context-switch overhead is negligible: the classic disadvantage of interrupts does not apply here.

EXERCISE 2 **Analysing an interrupt-driven system**

A computer receives data from a temperature sensor that produces one reading every second, and from a high-speed camera that produces 200 000 small frames per second.

1. Explain, step by step, what happens when the sensor has a reading ready.
2. Which device is well suited to interrupt-driven I/O, and which is not? Justify using the expression “interrupt storm”.
3. Suggest a more appropriate mechanism for the camera.

SOLUTION **Exercise 1**

1. The sensor’s controller stores the reading in its data register and raises its IRQ line; the interrupt controller forwards the request with the interrupt number; the CPU finishes its current instruction, saves the context of the running program on the stack, uses the interrupt vector table to find the sensor’s ISR, and jumps to it; the ISR reads the data register, stores the value in memory and acknowledges the interrupt; the context is restored and the suspended program resumes.
2. The sensor (one interrupt per second) is perfectly suited: the overhead is negligible and the CPU stays free between readings. The camera is not: at 200 000 interrupts per second, the CPU would spend nearly all its time saving and restoring contexts instead of processing images — an interrupt storm.
3. Direct Memory Access (DMA): the camera’s controller transfers whole blocks of frames directly into memory and interrupts the CPU only once per block, drastically reducing the number of interrupts.

KEY POINTS **Key points**

- An **interrupt** is a hardware signal sent by a device controller (via the IRQ line and the interrupt controller) to request the CPU’s attention.

- The CPU finishes its current instruction, saves the context, executes the **ISR** found through the **interrupt vector table**, then restores the context and resumes the program.
- The interrupt controller (PIC/APIC) manages several devices through masking and priorities.
- Interrupt-driven I/O frees the CPU during the device operation but keeps it involved in each transfer; the context-switch overhead makes it unsuitable for very fast devices (interrupt storms) — hence DMA, studied next.
- Distinguish hardware interrupts from software interrupts (system calls) and exceptions (division by zero, page fault).

4 DIRECT MEMORY ACCESS (DMA) AND COMPARISON OF MECHANISMS

In the previous section we saw that **interrupt-driven I/O** frees the CPU from the wasteful waiting loop of polling: the peripheral raises an interrupt when it is ready, and the CPU serves it only at that moment. This is an enormous improvement for slow devices such as keyboards and mice. But let us now push the reasoning one step further with a concrete situation.

EXPERIMENT **The cost of copying a file**

Suppose you copy a 4 GB video file from an external hard disk to the internal SSD of a computer in your school's computer laboratory. With interrupt-driven I/O, the disk controller raises an interrupt for every small unit of data (a few bytes or a few words) it has ready. The CPU must stop its current work, save its context, run the interrupt service routine, move the data, restore the context, and resume — millions of times. Even if each interrupt costs only a few microseconds of overhead, the accumulated cost becomes enormous: the CPU spends a large fraction of its time simply carrying data from the disk to memory, instead of running user programs. For large block transfers, even interrupts cost too much CPU time.

This observation motivates the third and most powerful control mechanism: **Direct Memory Access**.

4.1 Definition and Principle of DMA

DEFINITION **Direct Memory Access (DMA)**

Direct Memory Access (DMA) is a peripheral control mechanism in which data is transferred directly between a peripheral device and main memory, without passing through the CPU. The CPU only initiates the transfer (by programming a dedicated circuit) and is notified once at the end (by a single interrupt). During the whole block transfer, the CPU is free to execute other programs.

The intuition is simple. Polling is like a head teacher who walks to the photocopy room every minute to ask "Are the copies ready?" Interrupt-driven I/O is like telling the photocopy operator "Call me each time one page is ready" — better, but the head teacher is still interrupted for every single page. DMA is like saying: "Here is the stack of originals, here is where the copies must be placed, there are 500 pages — call me only when everything is finished." The head teacher delegates the whole job and works on something else in the meantime.

4.2 The DMA Controller (DMAC)

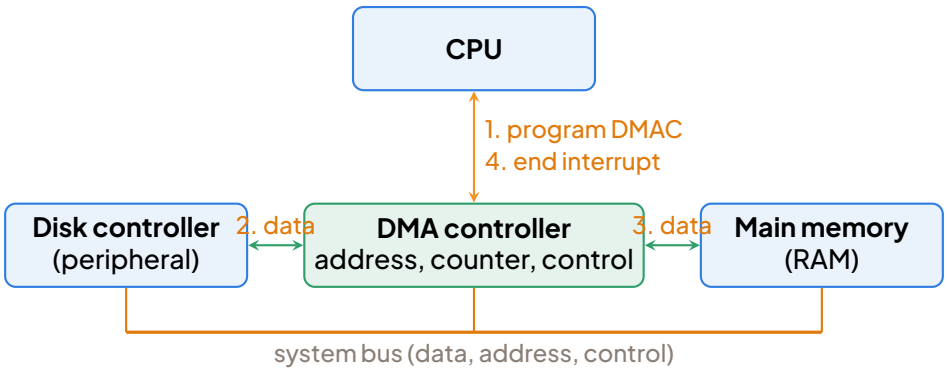
DMA is made possible by a specialised hardware circuit called the **DMA controller**.

DEFINITION **DMA controller and cycle stealing**

The **DMA controller (DMAC)** is a hardware circuit that manages a data transfer between a peripheral and main memory on behalf of the CPU. It contains in particular:

- an address register holding the source/destination address in memory (automatically incremented after each unit transferred);
- a transfer counter holding the number of data units still to be transferred (automatically decremented);
- a control register holding the transfer mode and direction (read from device or write to device), programmed by the CPU;
- bus control signals (bus request / bus grant) allowing it to take control of the system bus — a capability called **bus mastering**.

Cycle stealing is the technique by which the DMAC temporarily takes control of the system bus to transfer one unit of data, typically during bus cycles that the CPU is not using, and then gives the bus back. The DMAC "steals" bus cycles from the CPU, which may be slightly slowed down but is never stopped.



Block diagram of a DMA transfer: data flows directly between the disk controller and memory through the DMAC; the CPU intervenes only at the start (programming) and at the end (one interrupt).

4.3 Step-by-Step DMA Scenario

METHOD **How a DMA transfer proceeds**

A typical DMA transfer (for example, reading a block from disk into RAM) unfolds as follows:

1. Programming the DMAC. The CPU writes into the DMAC's registers: the memory address where data must be placed, the number of bytes to transfer, and the direction of transfer. The CPU also instructs the disk controller to prepare the data.
2. Delegation. The CPU is now free: it continues executing other programs while the transfer runs.
3. Block transfer by cycle stealing. For each unit of data, the DMAC requests the bus (bus request), receives the grant, transfers the data directly between the disk controller and memory, increments its address register and decrements its counter, then releases the bus.
4. Completion. When the counter reaches zero, the whole block has been transferred.
5. Final interrupt. The DMAC raises one single interrupt to inform the CPU that the transfer is complete. The CPU verifies the status and continues.

PROPERTY **CPU involvement under DMA**

Under DMA, the CPU's involvement in a transfer is reduced to two moments only: the initialisation of the DMAC (programming addresses, count and mode) and the treatment

of the single final interrupt. All the intermediate data movements are performed by the DMAC without any CPU intervention. (This property is directly examinable; be able to state and justify it.)

WARNING **DMA does not mean "no CPU at all"**

A very common mistake in the GCE examination is to write that "DMA transfers data without the CPU" and conclude that the CPU plays no role whatsoever. This is false. Without the CPU, the DMAC would not know where to read, where to write, or how much to transfer. The CPU must configure the DMAC before the transfer and supervise its completion (checking for errors via the final interrupt). The correct statement is: DMA transfers data without passing through the CPU's registers and without executing CPU instructions for each data unit — but the CPU initiates and supervises the operation.

4.4 Typical Users of DMA

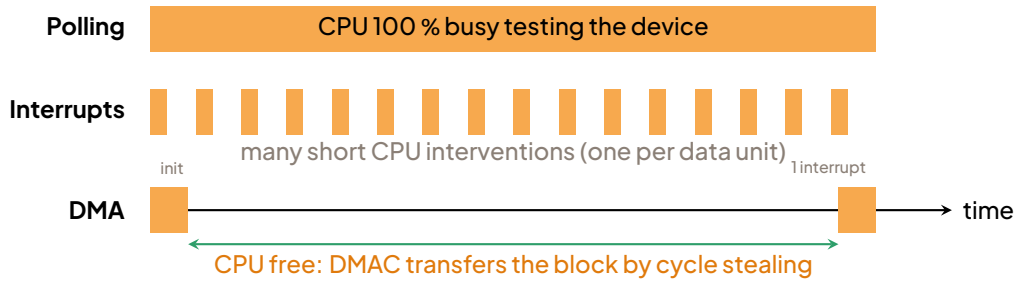
DMA is used by all peripherals that move **large blocks of data at high speed**:

- **Hard disks and SSDs**: loading programs and files into RAM (e.g. opening a large spreadsheet);
- **Sound cards**: streaming continuous audio to or from memory without gaps;
- **Graphics cards**: moving images and textures between RAM and video memory;
- **Network cards**: transferring received packets directly into memory buffers — essential when, for example, a server of a mobile money platform handles thousands of transactions per second.

4.5 Comparative Synthesis: Polling, Interrupts and DMA

EXEMPLE **The same transfer under the three mechanisms**

Consider transferring a 64 KB block from disk to RAM. Under polling, the CPU loops continuously, testing the disk status register: it is 100 % busy for the whole duration. Under interrupt-driven I/O, the CPU is interrupted for each small unit of data (say every 4 bytes, i.e. 16 384 interrupts): it can do other work between interrupts, but each interrupt costs context saving and restoring. Under DMA, the CPU programs the DMAC once, works freely during the transfer, and receives one interrupt at the end.



CPU occupation (orange) for the same block transfer under polling, interrupt-driven I/O and DMA. Under DMA the CPU is occupied only at the beginning and at the end.

Criterion	Programmed I/O (polling)	Interrupt-driven I/O	DMA
CPU involvement	Continuous: CPU tests the device in a loop	One interrupt per data unit	Initialisation + one final interrupt only
CPU availability	None during the transfer	Partial (works between interrupts)	Almost total during the transfer
Transfer speed	Slow, limited by the testing loop	Moderate, limited by interrupt overhead	High, at bus speed
Hardware cost	Very low (no extra hardware)	Low (interrupt controller)	Higher (DMA controller, bus mastering)
Suitable devices	Very slow, simple devices (keyboard in simple systems, sensors)	Slow to medium devices (keyboard, mouse, serial ports, printer)	Fast block devices (disks, SSDs, sound, graphics, network cards)

METHOD **Choosing the appropriate control mechanism for a device**

To answer an exam question asking which mechanism suits a given device, reason as follows:

1. Estimate the volume of data and the speed of the device: a few occasional bytes, or large continuous blocks?
2. If the device is very slow and produces rare data (a key press), polling or interrupts suffice; polling only if the system is extremely simple and has nothing else to do.
3. If the device produces data at unpredictable times and the CPU has other work, choose interrupt-driven I/O.
4. If the device transfers large blocks at high speed (disk, sound, video, network), choose DMA.

5. Justify your answer in terms of CPU time saved and hardware cost.

WORKED EXAMPLE 4 **Choosing and justifying a mechanism**

A school in Bamenda acquires (a) a mouse, (b) an SSD used to load the operating system, and (c) a temperature sensor read once per second by a small dedicated controller that does nothing else. For each device, state the most appropriate control mechanism and justify your choice.

SOLUTION

Solution.

- (a) Mouse — interrupt-driven I/O. The mouse produces small amounts of data at unpredictable times. Polling would waste CPU time between movements; DMA is unjustified for a few bytes. Interrupts let the CPU work and react immediately when the mouse moves.
- (b) SSD — DMA. Loading the OS means transferring very large blocks at high speed. Interrupt-per-unit would generate millions of interrupts; DMA transfers the blocks directly to RAM and raises a single interrupt per block, freeing the CPU.
- (c) Temperature sensor — programmed I/O (polling). The controller has no other task, so waiting in a loop costs nothing; the data rate (one value per second) is extremely low and no extra hardware (interrupt line or DMAC) is needed, which minimises cost.

4.6 Exam Extra: Buffered I/O and Spooling

DID YOU KNOW? **Beyond the hardware: buffering and spooling**

The three mechanisms above are hardware mechanisms. Operating systems build software techniques on top of them. With **buffered I/O**, data is temporarily stored in a memory area called a buffer so that a fast producer and a slow consumer can work at their own pace. **Spooling** (Simultaneous Peripheral Operations On-Line) generalises this idea: jobs for a slow device are queued on disk and sent one after another. The classic example is the print spooler: when several students in a cybercafé in Yaoundé print their documents at the same time, the operating system stores the jobs in a queue and feeds the printer progressively, while the users continue working. Expect occasional exam questions linking spooling to interrupt-driven I/O, since the printer signals the end of each job by an interrupt.

KEY POINTS **Key points of this section**

- DMA transfers data directly between a peripheral and memory, without passing through the CPU.
- The DMAC holds the memory address, the transfer counter and the control information, and takes control of the bus (bus mastering) by cycle stealing.
- The CPU only programs the DMAC at the start and receives one interrupt at the end.
- DMA is essential for fast block devices: disks, SSDs, sound, graphics and network cards.
- Polling = simple but CPU-wasting; interrupts = efficient for slow unpredictable devices; DMA = best for large fast transfers, at higher hardware cost.
- Buffering and spooling are OS techniques built on top of these hardware mechanisms.

EXERCISE 3 **Mechanisms under the microscope**

A network card receives 1 MB of data per second on average. (a) Explain why polling is unsuitable. (b) Explain why interrupt-driven I/O with one interrupt per byte would be unsuitable. (c) Describe how DMA solves the problem, stating precisely the role of the CPU.

SOLUTION **Exercise 1**

(a) Under polling the CPU would loop permanently testing the network card, wasting nearly all its time and risking to miss incoming data while busy elsewhere. (b) One interrupt per byte means about one million interrupts per second; each interrupt costs context saving, execution of the service routine and context restoring, so the overhead alone would saturate the CPU. (c) With DMA, the CPU programs the DMAC (memory buffer address, byte count, direction), then continues other work; the DMAC transfers the received data directly into memory by cycle stealing and raises a single interrupt when the buffer is full. The CPU's role is limited to initialising the transfer and processing the completed buffer on the final interrupt.

5 INTEGRATION ACTIVITY

5.1 Aims of the activity

By the end of this integration activity, you should be able to:

- **Analyse** the behaviour of real peripheral devices (keyboard, printer, hard disk, mouse) and **identify** the I/O control mechanism most likely used by each of them: **programmed I/O (polling)**, **interrupt-driven I/O** or **Direct Memory Access (DMA)**.
- **Justify** your diagnosis using two rigorous arguments: the degree of **CPU involvement** and the **speed** (data rate) of the peripheral.
- **Represent** each mechanism by a clear transfer diagram showing the CPU, memory, controller and device.
- **Propose and defend** a technical solution to a real performance problem (a printer that blocks the whole computer), by changing the control mechanism.
- **Communicate** your findings in a one-page diagnostic report, as a technician would do for a school administration.

This activity mobilises all the competences of the chapter: it is not enough to recite definitions — you must use them to explain observable facts and to make justified engineering decisions.

5.2 Situation

Diagnosing the computer room of GBHS Bamenda.

The Government Bilingual High School Bamenda has a computer room with 25 identical desktop computers used for ICT practicals and for the secretariat. Each machine has a 3.0 GHz processor, 4 GB of RAM, a 500 GB internal hard disk, a USB keyboard, an optical USB mouse, and runs a standard operating system. One old dot-matrix printer, connected to the secretary's computer, is shared by the whole room.

The ICT teacher, Mr. Ndi, has collected the following complaints and observations during the first term:

- **Observation A — the keyboard.** Students type exercises in a text editor. Even the fastest typist in Upper Sixth produces at most about 10 characters per second. Typing never slows the computer down, and no character is ever lost, even when the CPU is busy compiling a program at the same time.
- **Observation B — the dot-matrix printer.** Whenever the secretary prints a 30-page report, her computer becomes completely unusable: the mouse pointer freezes, no window responds, and she must wait until printing ends. The printer is slow (about 100 characters per second) and the printing of one page takes several minutes. The printer manual states that it has no internal buffer memory and that the computer sends each character "under direct control of the CPU".
- **Observation C — the hard disk.** A student copies a 2 GB video file from one partition of the hard disk to another. The transfer runs at about 100 MB/s and lasts roughly 20 seconds.

During the copy, the student continues to type a document and scroll in a web page without any noticeable slowdown. The CPU usage shown by the task manager stays below 10% during the whole copy.

- **Observation D — the mouse.** Students move the mouse constantly. The pointer follows instantly, and clicks are never missed, even while the hard disk copy of Observation C is running.

Mr. Ndi asks your class, as future computer scientists, to prepare a **diagnostic report** explaining why each device behaves as it does, in terms of the three peripheral control mechanisms studied in this chapter, and to propose a solution for the printer problem. The school administration will use your report to decide whether to buy a new printer or to reconfigure the existing one.

METHOD **How to diagnose a control mechanism**

When you must identify which mechanism a device uses, proceed in this order:

1. Estimate the data rate of the device (characters/s, MB/s) and compare it with the CPU speed.
2. Ask who moves the data: the CPU itself in a loop (polling), the CPU only when signalled (interrupt-driven I/O), or a dedicated controller (DMA).
3. Check the symptoms: a frozen computer during transfer suggests polling; a responsive computer with low CPU usage during a fast transfer suggests DMA.
4. Conclude by matching your evidence to one of the three mechanisms and justify it explicitly.

5.3 Tasks

Work in groups of three or four. Answer the following questions in order; each one builds on the previous.

1. **Recall.** For each of the three control mechanisms (programmed I/O, interrupt-driven I/O, DMA), state in two or three sentences: (i) who controls the transfer, (ii) what the CPU does during the transfer, (iii) for which category of device it is appropriate.
2. **Diagnose the keyboard (Observation A).** Which mechanism is used? Justify by comparing the keyboard's data rate (about 10 characters/s) with the CPU's speed, and by explaining why no character is lost even when the CPU is busy.
3. **Diagnose the printer (Observation B).** The manual says each character is sent "under direct control of the CPU" and the printer has no buffer. Which mechanism is this? Using the figures given (100 characters/s, a 30-page document), explain precisely why the whole computer freezes during printing.
4. **Diagnose the hard disk (Observation C).** Which mechanism explains that a 2 GB copy at 100 MB/s runs while the CPU usage stays below 10%? Describe the role of the DMA con-

troller, the bus, and the single interrupt at the end of the transfer.

5. **Diagnose the mouse (Observation D).** Which mechanism guarantees that clicks are never missed, even during the disk copy? Why would pure polling be risky here?
6. **Draw.** For each of the four devices, draw the transfer diagram corresponding to its mechanism: show the CPU, the main memory, the device controller (and the DMA controller where relevant), the data path and the interrupt line.
7. **Propose a fix.** The administration hesitates between two solutions for the printer: (S1) replace it by a modern printer with a large internal buffer and interrupt-driven (or DMA) transfer; (S2) keep the printer but change the way the operating system sends data, using interrupts and a software buffer (spooling). For each solution, explain the mechanism after the fix, the effect on the secretary's computer, and one advantage and one drawback. Conclude with a justified recommendation.
8. **Report.** Assemble your answers into a one-page diagnostic report addressed to the principal: a short table summarising the four diagnoses, one diagram, and your recommendation for the printer.

WARNING **Common traps to avoid**

- Do not say "the printer is slow, so it uses polling" as a definition: polling is a mechanism, not a consequence of slowness. The correct argument is that the CPU actively waits for the device, so a slow device under polling monopolises the CPU.
- DMA does not mean "no CPU at all": the CPU initiates the transfer and is interrupted at the end; only the data movement itself is handled by the DMA controller.
- Interrupt-driven I/O still consumes CPU time for every unit of data transferred; for a very fast device (100 MB/s), interrupt-per-byte would be impossible — that is exactly why DMA exists.
- Do not confuse the device controller (the electronic interface attached to one device) with the DMA controller (which manages block transfers over the bus); a DMA controller is a specialised controller that can take charge of the system bus.

5.4 Model answer

Question 1 — Summary of the three mechanisms.

Mechanism	Who controls	CPU during transfer	Appropriate for
Programmed I/O (polling)	CPU executes a loop that repeatedly tests the device status register and moves each data word itself.	Fully busy: it waits actively and transfers every word.	Very simple, slow devices; predictable transfers.
Interrupt-driven I/O	CPU starts the operation; the controller signals with an interrupt when ready; the ISR transfers the data.	Free between interrupts; busy only in short interrupt service routines.	Slow, unpredictable, event-driven devices (keyboard, mouse).
DMA	DMA controller moves whole blocks directly between device and memory; CPU only configures and is interrupted at the end.	Almost free: one interrupt per block, not per word.	Fast devices transferring large blocks (disk, sound, network).

Question 2 — Keyboard: interrupt-driven I/O. A keyboard produces at most about 10 characters per second, at unpredictable instants. Polling would waste CPU cycles testing a device that is almost always idle, and a polling loop busy elsewhere could miss a keystroke. With interrupt-driven I/O, the keyboard controller raises an **interrupt request** at each key press; the CPU suspends its current work for a few microseconds, the interrupt service routine reads the scan code into a buffer, and normal work resumes. This explains Observation A perfectly: zero characters lost, zero perceptible slowdown, even while compiling.

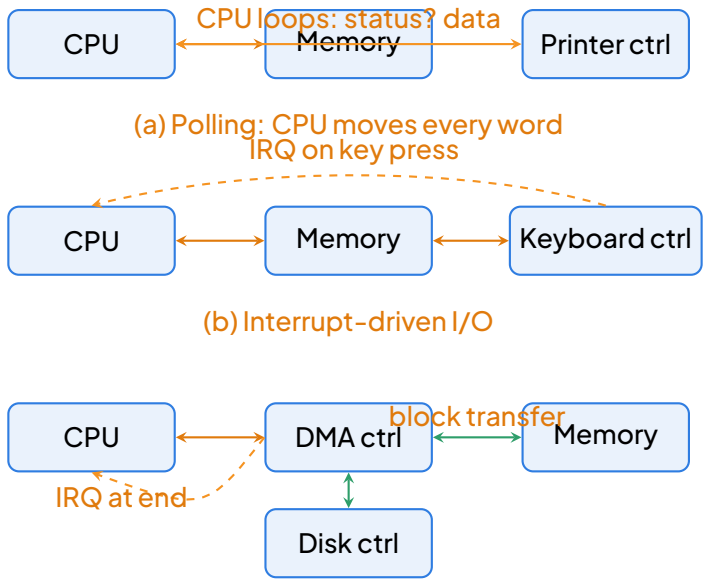
Question 3 — Printer: programmed I/O (polling). "Each character sent under direct control of the CPU" with no buffer is the signature of **programmed I/O**: for every character, the CPU loops reading the printer's status register until "ready", then writes the character. At 100 characters/s, each character keeps the CPU waiting about $1/100 = 0.01$ s — an eternity for a 3.0 GHz processor, which executes about 3×10^9 clock cycles per second, i.e. roughly 3×10^7 clock cycles wasted per character. A 30-page report contains tens of thousands of characters, so the CPU spends essentially all its time waiting for the printer: the mouse pointer freezes and no window responds. The computer is not broken; it is busy waiting.

Question 4 — Hard disk: DMA. Copying 2 GB at 100 MB/s means about 2×10^9 bytes moved in 20 s, i.e. about 10^8 bytes per second. Interrupt-per-byte would require 10^8 interrupts per second — physically impossible, since servicing one interrupt already costs thousands of clock cycles. Instead, the CPU programs the **DMA controller** (source address, destination address, byte count) and continues other work; the controller takes charge of the bus and moves the data directly between the disk buffer and memory, using **cycle stealing** when the CPU also needs the bus. Only at the end of each block does the controller raise a single interrupt. Hence

CPU usage below 10% and a perfectly responsive machine: Observation C is a textbook case of DMA.

Question 5 — Mouse: interrupt-driven I/O. Mouse movements and clicks are unpredictable, low-volume events that must be serviced immediately for the pointer to feel responsive. Each movement or click generates an interrupt; the ISR updates the pointer position or queues the click. Pure polling would risk missing a fast click if the polling interval were too long, or wasting CPU time if it were too short. Interrupts give both responsiveness and efficiency, even during the DMA disk copy, because DMA leaves the CPU free to service interrupts.

Question 6 — Diagrams. The three configurations are summarised below. The keyboard and the mouse both correspond to diagram (b); the printer to diagram (a); the hard disk to diagram (c).



Transfer diagrams for the three control mechanisms. Solid arrows are data paths; dashed arrows are interrupt request (IRQ) lines.

Question 7 — Fixing the printer.

- **S1 (new printer with buffer + interrupt/DMA transfer).** The computer writes a whole page into the printer's buffer at high speed, the printer signals readiness by interrupt, and the CPU is free between transfers. The secretary's computer stays responsive. Advantage: definitive, fast, also better print quality. Drawback: cost of new equipment.
- **S2 (spooling + interrupt-driven output).** The operating system copies the document into a software buffer on disk (the **spool**), returns control immediately, and a background process sends characters to the printer as it becomes ready, using interrupts. Advantage: no hardware cost. Drawback: printing remains slow (100 characters/s) and needs a driver supporting interrupts.

Recommendation: adopt S2 immediately as a zero-cost fix, and plan S1 in the next budget,

since the old printer's speed will remain a bottleneck for a whole school.

Question 8 — Report. The one-page report contains the summary table below, diagram (c), and the recommendation above.

Device	Mechanism	Key evidence
Keyboard	Interrupt-driven I/O	Very slow (10 char/s), unpredictable; no character lost while CPU busy.
Printer	Programmed I/O (polling)	CPU sends each character directly; no buffer; computer frozen during printing.
Hard disk	DMA	100 MB/s block transfer with CPU usage below 10%.
Mouse	Interrupt-driven I/O	Instant response, clicks never missed, even during disk copy.

EXERCISE 4 **Going further**

The school buys a network card that receives data at 100 Mbit/s. Explain why programmed I/O and even interrupt-per-byte I/O are unsuitable, and describe the mechanism the card must use.

SOLUTION **Going further**

At 100 Mbit/s, a byte (8 bits) arrives every 8×10^{-8} s: the CPU can neither poll fast enough nor service an interrupt per byte, since each interrupt service routine alone costs far more than 8×10^{-8} s. The card must use **DMA**: its controller assembles incoming data into packets in an internal buffer, then transfers whole packets directly into main memory, raising one interrupt per packet (or per batch of packets). The CPU only processes complete packets, keeping the system responsive.

6 METHODS AND WORKED EXAMPLES

6.1 Method 1 — Identifying the control mechanism used in a given situation

METHOD **Recognising polling, interrupt-driven I/O and DMA**

To identify which peripheral control mechanism is described in an exercise or a real system, proceed as follows:

1. Ask: who initiates and supervises the transfer? If the CPU repeatedly tests a status flag in a loop until the device is ready, it is **programmed I/O** (polling).
2. Ask: does the device signal the CPU? If the peripheral raises an interrupt request line (IRQ) and the CPU suspends its current program to run an interrupt service routine (ISR), it is **interrupt-driven I/O**.
3. Ask: who moves the data? If a special circuit, the DMA controller (DMAC), transfers data directly between the device and main memory while the CPU only starts and ends the operation, it is **DMA**.
4. Check the size and speed of the transfer: small, slow, unpredictable transfers (keyboard, mouse) suggest polling or interrupts; large, fast, block transfers (disk, sound card, network interface) suggest DMA.
5. Conclude in one sentence naming the mechanism and justifying it with one observable clue from the statement.

Reflex: polling = CPU waits and wastes time; interrupt = device calls the CPU; DMA = a third party (DMAC) does the moving.

WORKED EXAMPLE 5 **Identifying mechanisms in a Cameroonian cybercafé**

In a cybercafé in Buea, the system administrator observes the following: (a) when a customer presses a key, the keyboard controller sends a signal that makes the CPU momentarily leave the word processor to read the character; (b) when a 500 MB video file is copied from an external hard disk to the internal drive, the CPU remains almost free and customers can keep working; (c) a small program controlling a temperature sensor checks the sensor's status register every second in a loop. For each case, name the I/O control mechanism and justify.

SOLUTION

Solution.

1. Case (a) — keyboard: The device signals the CPU, which suspends its current task and

executes a service routine to read the character. This is interrupt-driven I/O. Justification: the CPU does not loop waiting; the peripheral initiates the contact via an interrupt request.

2. Case (b) — disk copy of 500 MB: The transfer is large and the CPU stays free, meaning a DMA controller moves the data directly between the disk controller and main memory. This is Direct Memory Access (DMA). Justification: the CPU only configures the transfer (source, destination, length) and is interrupted once at the end; the bulk transfer does not occupy it.
3. Case (c) — temperature sensor: The program repeatedly tests a status register in a loop until data is ready. This is programmed I/O (polling). Justification: the CPU itself initiates and supervises every step and wastes cycles between checks.

Conclusion: (a) interrupt-driven I/O, (b) DMA, (c) polling.

6.2 Method 2 — Describing the steps of programmed I/O (polling)

METHOD **Writing the polling sequence**

When asked to describe or draw the flowchart of programmed I/O:

1. The CPU issues a command to the device controller (e.g. “read a byte”).
2. The CPU reads the status register of the controller.
3. Test the ready/busy flag: if the device is busy, loop back to step 2 (this waiting loop is the weakness of polling).
4. When the flag shows ready, the CPU transfers one data unit (byte/word) between the controller’s data register and memory (or a CPU register).
5. Repeat steps 2–4 for each unit until the whole block is transferred, then check for errors and terminate.

Key idea to remember: the CPU is fully occupied during the whole transfer; polling is simple and needs no extra hardware, but it wastes CPU time — acceptable only for very simple or dedicated systems.

WORKED EXAMPLE 6 **Polling a printer**

A dot-matrix printer controller has a status register whose bit 0 is 1 when the printer is ready to accept a character, and 0 when it is busy. A program must print the 4-character word “EXAM”. (a) Write, in numbered steps or pseudocode, the polling algorithm. (b)

State two disadvantages of this method for a busy school computer room.

SOLUTION

Solution.
(a) Polling algorithm:

1. For each character *c* of "EXAM" (E, X, A, M):
   (a) Read the printer's status register.
   (b) Test bit 0: if it equals 0 (busy), go back to step (i) — the CPU waits in the loop.
   (c) If bit 0 equals 1 (ready), write *c* into the printer's data register; the controller prints it and clears the ready flag.
2. After the last character, check the error bits of the status register; report any fault, else end.

Pseudocode:

```
FOR each character c in "EXAM"
  REPEAT read STATUS UNTIL (STATUS AND 1) = 1
  WRITE c to DATA register
CHECK errors; STOP
```

(b) Disadvantages:

- The CPU is blocked in the waiting loop while the slow printer is busy: it cannot serve other users or programs, so the whole computer room slows down.
- CPU time (and therefore electrical energy) is wasted on useless status tests; the method does not scale when several peripherals are connected.

6.3 Method 3 — Describing the interrupt-driven I/O sequence

METHOD **Writing the interrupt sequence**

To describe interrupt-driven I/O, always mention these steps in order:

1. The CPU starts the device with a command, then continues executing another program (no waiting loop).
2. When ready, the device controller raises an interrupt request (IRQ) on an interrupt line.
3. At the end of the current instruction, the CPU detects the interrupt, saves the context (program counter, status register, general-purpose registers) on the stack.
4. The CPU uses the interrupt vector to find and execute the ISR (Interrupt Service Rou-

tine), which performs the data transfer.

5. On the `return from interrupt` instruction, the context is restored and the interrupted program resumes exactly where it stopped.

Reflexes: interrupts can be maskable (can be disabled by software) or non-maskable (NMI, for critical events like power failure); several devices are managed with priorities and vectoring. The CPU is free between interrupts but still executes every byte/word transfer itself.

WORKED EXAMPLE 7 **Interrupt-driven keyboard handling**

A student types on a keyboard at an average speed of 4 characters per second while the computer compiles a program. Each character typed triggers an interrupt whose service routine takes 20 μs. (a) Explain, step by step, what happens in the machine when a key is pressed. (b) Calculate the fraction of CPU time consumed by keyboard interrupts. (c) Conclude on the suitability of interrupts for this device.

SOLUTION

Solution.
(a) Sequence of events:

1. The CPU is executing the compiler. The keyboard controller was previously enabled to interrupt (its interrupt mask bit is set).
2. A key is pressed: the controller stores the scan code in its data register and raises an IRQ line.
3. After finishing the current machine instruction, the CPU acknowledges the interrupt, pushes the program counter and status register onto the stack (context saving).
4. The interrupt vector gives the address of the keyboard ISR; the ISR reads the scan code from the controller's data register, converts it to ASCII, and places the character in a keyboard buffer.
5. The ISR ends with a return-from-interrupt (RTI/IRET): the saved context is popped and the compiler continues as if nothing had happened.

(b) Fraction of CPU time:
Interrupts per second: 4. Time per interrupt: 20 μs = 20×10^{-6} s.

$$\text{CPU fraction} = 4 \times 20 \times 10^{-6} = 8 \times 10^{-5} = 0.00008 = 0.008\,\%.$$

(c) Conclusion: Only 0.008 % of CPU time is used: interrupt-driven I/O is perfectly suited to slow, unpredictable devices like keyboards, because the CPU remains almost entirely free for useful work and no polling loop is wasted.

6.4 Method 4 — Computing CPU overhead of polling

METHOD **Quantifying the cost of polling**

To compute how much CPU time polling wastes:

1. Identify the device data rate D (bytes/s) and the time of one poll t_p (a status read + test + branch).
2. Compute the number of polls per second: if the CPU polls exactly once per data unit, $N = \frac{D}{s}$ where s is the size of one data unit; if the statement gives a fixed polling frequency f, then $N = f$.
3. CPU time per second: $T = N \times t_p$; express as a percentage: $\frac{T}{1\,\text{s}} \times 100\,\%$.
4. Compare with the useful work: conclude whether polling is acceptable.

Key formulas: $N = \frac{D}{s}$ (polls needed per second); overhead % $= N \times t_p \times 100\,\%$ (with t_p in seconds).

WORKED EXAMPLE 8 **Cost of polling a serial link**

A serial port receives data at 9 600 bits/s, organised in bytes of 8 bits (ignore start/stop bits). To read it by polling, the CPU executes a status check taking 2 μs. Assume the CPU polls exactly once per received byte, plus it polls at a fixed rate of 100 000 polls per second to be sure not to miss data. (a) Compute the byte rate. (b) Compute the percentage of CPU time used by polling at 100 000 polls/s. (c) Comment.

SOLUTION

Solution.
(a) Byte rate:

$$D = \frac{9\,600\ \text{bits/s}}{8\ \text{bits/byte}} = 1\,200\ \text{bytes/s}.$$

(b) CPU percentage with fixed polling rate:
Number of polls per second: $N = 100\,000$. Time per poll: $t_p = 2\,\mu\text{s} = 2 \times 10^{-6}$ s.

$$T = N \times t_p = 100\,000 \times 2 \times 10^{-6} = 0.2\ \text{s per second}.$$

$$\text{Overhead} = \frac{0.2}{1} \times 100\,\% = 20\,\%.$$

(c) Comment: Although the useful data represents only 1 200 bytes/s (needing at best 1 200 useful polls, i.e. 0.24 % of CPU), the blind fixed-rate polling consumes 20 % of the CPU — nearly all of it testing a device that has nothing to deliver. This illustrates the fundamental weakness of programmed I/O: the CPU pays for waiting, not for transferring.

An interrupt-driven design would reduce the cost to about $1200 \times t_{ISR}$ per second, i.e. a fraction of a percent.

6.5 Method 5 — Describing and computing a DMA transfer

METHOD **DMA sequence and transfer-time calculations**

Sequence to describe:

1. The CPU programs the DMA controller: source/destination addresses, number of bytes, direction, then issues the start command to the device.
2. The DMAC requests the bus; the CPU grants it (bus grant) and the DMAC performs the transfer directly between device and memory, often by cycle stealing (using bus cycles while the CPU is not using them) or in burst mode.
3. When the count reaches zero, the DMAC sends one single interrupt to the CPU to signal completion.

Calculations:

- Transfer time (device side): $T = \dfrac{\text{block size}}{\text{device rate}}$.
- CPU involvement: only the setup time t_{setup} plus one end-of-transfer ISR t_{ISR}, whatever the block size.
- Compare with interrupt-driven I/O: one ISR per unit transferred.

WORKED EXAMPLE 9 **DMA versus interrupts for a disk block**

A hard disk transfers a block of 64 KB (1 KB = 1024 bytes) to RAM at a sustained rate of 20 MB/s (1 MB = 1024 KB). With interrupt-driven I/O, each 4-byte word transferred costs one interrupt of 5 μs. With DMA, setup takes 10 μs and the final interrupt takes 5 μs. (a) Describe the DMA transfer step by step. (b) Compute the CPU time used by each method. (c) Compute the transfer duration on the device side and conclude.

SOLUTION

Solution.
(a) DMA steps:

1. The CPU writes into the DMAC registers: memory address of the buffer, count = 64 KB, direction "disk → memory", then commands the disk to read.
2. The disk controller signals the DMAC when data is available; the DMAC takes control of

the system bus (cycle stealing) and moves the words directly into RAM, incrementing the address and decrementing the counter.

3. The CPU continues executing other programs whenever it does not need the bus.
4. When the counter reaches 0, the DMAC raises one interrupt; the ISR checks status and informs the operating system that the block is ready.

(b) CPU time:

Number of words: $\frac{64 \times 1\,024 \text{ bytes}}{4 \text{ bytes/word}} = 16\,384$ words.

- Interrupt-driven I/O: $16\,384 \times 5\ \mu s = 81\,920\ \mu s \approx 81.9$ ms of CPU time.
- DMA: $10\ \mu s + 5\ \mu s = 15\ \mu s$ of CPU time.

(c) Transfer duration:

$$T = \frac{64 \text{ KB}}{20 \text{ MB/s}} = \frac{64}{20 \times 1\,024} \text{ s} \approx 3.125 \text{ ms}.$$

Conclusion: The disk delivers the block in about 3.1 ms, but interrupt-driven I/O would need 81.9 ms of CPU service — the CPU cannot even keep up with the disk, and the transfer would be slowed down. DMA uses only 15 μs of CPU time, so the transfer runs at full device speed and the CPU remains free. This is why DMA is mandatory for fast block devices.

6.6 Method 6 — Choosing the appropriate mechanism for a device

METHOD **Justifying the choice of a control mechanism**

To answer "which mechanism is most appropriate and why?":

1. Estimate the data volume per transfer and the device speed.
2. Estimate the predictability of arrivals (regular stream vs. random events).
3. Apply the decision rule:
   - Very simple system, single task, slow device → polling (simplest hardware/software).
   - Slow device, unpredictable events, small units (keyboard, mouse, serial port) → interrupt-driven I/O.
   - Fast device, large blocks (disk, SSD, sound card, network card, graphics) → DMA, usually combined with one interrupt at the end.
4. Justify with a quantitative argument (CPU overhead) whenever possible.

Reflex: in real computers the three mechanisms coexist; the OS chooses per device.

WORKED EXAMPLE 10 **Equipping a school server in Yaoundé**

A school server in Yaoundé manages: (i) a console keyboard used occasionally by the administrator; (ii) a network card receiving bursts of 1 MB files from 40 student computers; (iii) a small microcontroller-based fire alarm sensor that must be checked regularly and whose reading program is the only task of its dedicated microcontroller. For each device, choose a control mechanism and justify.

SOLUTION

Solution.

- (i) Console keyboard — interrupt-driven I/O. Keystrokes are rare, small (1 byte) and unpredictable. Polling would waste CPU permanently; DMA is pointless for single bytes. An interrupt per keystroke costs a few microseconds only when a key is actually pressed.
- (ii) Network card receiving 1 MB files — DMA (with a completion interrupt). The transfers are large blocks arriving at high speed. Interrupt-per-word would generate hundreds of thousands of interrupts per file and saturate the CPU; DMA moves each file directly into memory buffers while the CPU keeps serving the 40 students, and one interrupt per completed buffer suffices.
- (iii) Fire alarm sensor on a dedicated microcontroller — programmed I/O (polling). The microcontroller has a single task, so "wasting" its time in a polling loop costs nothing: there is no other work to do. Polling is the simplest and most predictable solution, and the regular checking matches a periodic sensor reading. (An interrupt on the alarm line can be added for the emergency event itself.)

Conclusion: the choice always balances transfer size, device speed, event predictability and CPU availability; there is no universally best mechanism.

6.7 Method 7 — Comparing the three mechanisms in an examination table or essay

METHOD **Building a rigorous comparison**

When asked to compare programmed I/O, interrupt-driven I/O and DMA:

1. Compare along fixed criteria: who initiates the transfer, who moves the data, CPU involvement, hardware cost, complexity, typical devices, reaction speed.
2. Give one advantage and one limitation for each mechanism.
3. Finish with a synthesis sentence: modern systems combine all three (e.g. DMA for the disk, interrupts for the keyboard, polling inside some drivers).

Key ideas: polling wastes CPU in waiting loops; interrupts remove the waiting but the CPU

still moves every unit; DMA frees the CPU from the transfer itself at the price of an extra controller and bus arbitration.

WORKED EXAMPLE 11 **GCE-style comparison question**

(a) Construct a table comparing programmed I/O, interrupt-driven I/O and DMA according to: initiator of the transfer, data mover, CPU utilisation, hardware complexity, and typical applications. (b) State one advantage and one disadvantage of each mechanism. (c) Explain why a modern smartphone uses all three mechanisms simultaneously.

SOLUTION

Solution.
(a) Comparison table:

Criterion	Programmed I/O	Interrupt-driven	DMA
Initiator	CPU (polls status)	Device (IRQ)	CPU starts, DMAC runs
Data mover	CPU	CPU (in the ISR)	DMA controller
CPU utilisation	Very high (waiting loop)	Low between events	Negligible (setup + end)
Hardware complexity	Minimal	Interrupt controller	DMAC + bus arbitration
Typical uses	Simple embedded systems	Keyboard, mouse, serial	Disk, network, sound, GPU

(b) Advantage / disadvantage:

- Programmed I/O: advantage — simplest, no extra hardware, fully predictable timing; disadvantage — CPU is blocked and wastes time while the device is busy.
- Interrupt-driven I/O: advantage — CPU does useful work until the device calls, good reactivity; disadvantage — one interrupt per data unit, so overhead explodes for fast block devices, and context saving costs time.
- DMA: advantage — fastest bulk transfer, CPU almost totally free; disadvantage — extra hardware (DMAC), bus contention with the CPU, more complex to program.

(c) Smartphone: A smartphone is a multi-device system with heterogeneous needs: its touchscreen and keypad generate rare, unpredictable events handled by interrupts; its flash storage, camera and mobile network (4G/5G modem) move large data blocks using DMA so that applications stay responsive; and some simple drivers or boot code use polling loops where a dedicated task waits on a status flag. Combining the three mechanisms lets each peripheral be served with the best trade-off between CPU load, speed and hardware cost.

WARNING **Frequent mistakes at the GCE**

- Confusing who initiates with who transfers: in DMA the CPU initiates but the DMAC transfers.
- Saying interrupts "cost nothing": every interrupt costs a context save/restore and an ISR execution.
- Forgetting that polling is not "wrong": it is the right choice for dedicated single-task systems.
- Mixing units in calculations: always convert μs to seconds and KB to bytes (1 KB = 1 024 bytes) before computing percentages.

7 EXERCISES

7.1 I check my knowledge

EXERCISE 5 **Multiple-choice quiz**

For each question, choose the single correct answer.

1. During **programmed I/O (polling)**, the CPU repeatedly reads which element of the device controller?

   (a) the data buffer register

   (b) the status register

   (c) the instruction register of the CPU

   (d) the memory address register (MAR)

2. At the end of a block transfer, the DMA controller signals the CPU by raising:

   (a) a clock pulse

   (b) a bus request

   (c) an interrupt request (IRQ)

   (d) a parity bit

3. The software component that translates operating system commands into commands understood by a specific peripheral is:

   (a) the compiler

   (b) the device driver

   (c) the bootstrap loader

   (d) the interrupt vector

4. Which mechanism transfers data between a peripheral and main memory without passing each word through the CPU?

   (a) polling

   (b) interrupt-driven I/O

   (c) DMA

   (d) spooling

EXERCISE 6 **True or false — justify**

State whether each statement is true or false, and justify your answer in one or two sentences.

1. In polling, the CPU can execute other useful programs while it waits for a slow peripheral to become ready.
2. A device controller is a hardware circuit, whereas a device driver is software.
3. In interrupt-driven I/O, the peripheral itself informs the CPU when it needs attention.
4. DMA is the cheapest control mechanism to implement in terms of hardware.

EXERCISE 7 **Definitions**

Define each of the following terms in at most three lines:

1. peripheral device;
2. device controller (interface);
3. interrupt service routine (ISR);
4. bus mastering (cycle stealing) in DMA.

EXERCISE 8 **Direct calculation**

A keyboard produces one character every 200 ms on average. The CPU executes 2×10^9 instructions per second, and checking the keyboard status register by polling costs 5 instructions per check, performed every 1 ms.

1. How many status checks does the CPU perform between two successive keystrokes?
2. How many instructions are consumed by polling between two keystrokes?
3. What fraction of the CPU time is used by polling, and what fraction of the checks is unsuccessful? Comment.

7.2 I practise

EXERCISE 9 **Sequencing a DMA transfer**

The steps of a DMA block transfer from a hard disk to main memory are given below in dis-

order:

a. The DMAC raises an interrupt to signal the end of the transfer.

b. The CPU programs the DMAC with the source address, the destination address and the number of words.

c. The DMAC transfers the data block directly to memory, stealing bus cycles.

d. The CPU initiates the disk read command and switches to another task.

e. The CPU verifies the transfer status and resumes normal control of the bus.

Copy these steps in the correct chronological order and justify each position in one sentence.

EXERCISE 10 **Completing a comparison table**

Copy and complete the following table comparing the three I/O control mechanisms. For the last row, give two suitable example devices for each mechanism (think of a keyboard, a mouse, a printer, a hard disk, a network card, a flash drive).

Criterion	Polling	Interrupt-driven I/O	DMA
CPU involvement during transfer			
Relative speed for large blocks			
Hardware cost / complexity			
Two example devices			

EXERCISE 11 **Scenario analysis: the frozen computer**

Manka'a inserts a faulty USB flash drive into her computer in Bamenda. Immediately, the mouse pointer stops moving and the whole machine appears frozen for several seconds.

1. Which I/O control mechanism is most likely involved in this behaviour?
2. Explain, step by step, why the CPU is blocked and cannot serve other peripherals.
3. Suggest one hardware or software feature that would prevent a single faulty device from freezing the whole system.

EXERCISE 12 **Choosing the right mechanism**

For each of the following situations, state the most appropriate control mechanism (polling, interrupt-driven I/O or DMA) and justify your choice:

1. a point-of-sale terminal in a Douala supermarket reading keys pressed by the cashier;
2. copying a 4 GB video file from an external hard disk to the internal disk;
3. a simple washing-machine microcontroller checking every millisecond whether the door is closed;
4. a network interface card receiving a continuous stream of packets at high speed.

7.3 I prepare for the GCE

EXERCISE 13 **Structured question: from key press to memory (10 marks)**

1. Define the terms device controller and device driver, clearly stating for each one whether it is hardware or software. (4 marks)
2. Using a clearly labelled diagram, explain how a character typed on a keyboard reaches main memory using interrupt-driven I/O. Your answer must mention: the status and data registers of the controller, the interrupt request line, the interrupt service routine, and the role of the device driver. (6 marks)

EXERCISE 14 **Essay: a compromise between polling and DMA (15 marks)**

"Interrupt-driven I/O is a compromise between programmed I/O (polling) and Direct Memory Access."
Discuss this statement. Your essay should:

1. briefly recall the operating principle of each of the three mechanisms;
2. present two advantages and one limitation of each mechanism;
3. conclude by justifying why modern personal computers use all three mechanisms simultaneously, illustrating with concrete devices (keyboard, printer, hard disk, network card).

EXERCISE 15 **Problem: performance study of a data transfer (15 marks)**

A cybercafé in Yaoundé backs up 64 MB of data from a hard disk to memory every evening. Take 1 MB = 2^{20} bytes. The disk controller delivers data in words of 4 bytes. The CPU executes instructions at 10^9 instructions per second.

1. Compute the number of words to be transferred. (2 marks)
2. With programmed I/O, transferring each word costs the CPU 40 instructions (status check plus data movement). Calculate the total CPU time needed for the backup. (4 marks)
3. With interrupt-driven I/O, each word still costs 40 instructions, but an overhead of 20 instructions per interrupt is added. Calculate the new total CPU time and comment. (4 marks)
4. With DMA, the CPU only spends 5 000 instructions to program the DMAC and 2 000 instructions to handle the single end-of-transfer interrupt. Calculate the CPU time used and the percentage of CPU time saved compared with polling. (5 marks)

8 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — Multiple-choice quiz**

1. (b) the status register. Polling consists precisely in looping on the ready/busy flag of the controller's status register until the device signals that it is ready.
2. (c) an interrupt request (IRQ). The DMAC interrupts the CPU only once, when the whole block has been transferred (word count = 0).
3. (b) the device driver. It is the software layer between the operating system and the hardware controller, translating generic OS commands into device-specific orders.
4. (c) DMA. The DMA controller moves data directly between the peripheral and memory, becoming temporary bus master, so words never transit through the CPU.

SOLUTION **Exercise 2 — True or false — justify**

1. False. In polling the CPU is stuck in a waiting loop, repeatedly testing the status flag; it performs no useful work while the device is not ready. This is called busy waiting.
2. True. The controller (interface adapter) is an electronic circuit placed between the system bus and the device; the driver is a program executed by the CPU, usually part of the operating system.
3. True. The controller raises an interrupt request line; the CPU suspends its current task, saves its context and runs the interrupt service routine before resuming.
4. False. DMA requires an additional hardware component, the DMA controller (DMAC), capable of bus mastering; it is the most complex and most expensive of the three mechanisms. Polling is the cheapest.

SOLUTION **Exercise 3 — Definitions**

1. Peripheral device: any external equipment connected to the computer to perform input (keyboard), output (printer), storage (disk) or communication (network card).
2. Device controller: the hardware interface circuit between a peripheral and the system bus; it holds status, control and data registers and supervises the physical operation of the device.

3. Interrupt service routine (ISR): the section of code (supplied by the device driver) executed by the CPU in response to an interrupt, in order to service the requesting device before normal execution resumes.
4. Bus mastering (cycle stealing): the DMAC temporarily takes control of the system bus to transfer words directly between the device and memory, "stealing" bus cycles from the CPU, which simply waits when it needs the bus.

SOLUTION **Exercise 4 — Direct calculation**

1. Between two keystrokes, the number of status checks is

$$\frac{200\ \text{ms}}{1\ \text{ms}} = 200 \text{ checks.}$$

2. Instructions consumed by polling between two keystrokes:

$$200 \times 5 = 1\,000 \text{ instructions.}$$

3. In 200 ms = 0.2 s the CPU could execute

$$2 \times 10^9 \times 0.2 = 4 \times 10^8 \text{ instructions.}$$

The fraction of CPU time used by polling is therefore

$$\frac{1\,000}{4 \times 10^8} = 2.5 \times 10^{-6} \approx 0.00025\,\%.$$

Out of 200 checks, only the last one finds a character, so the fraction of unsuccessful checks is

$$\frac{199}{200} = 99.5\,\%.$$

Comment: polling a very slow device wastes almost all the checks (99.5 % find nothing), even though the absolute CPU cost remains small here because the keyboard is so slow. With several devices polled frequently, or with a faster device, the waste becomes significant — this is exactly why interrupts were invented.

SOLUTION **Exercise 5 — Sequencing a DMA transfer**

Correct chronological order: b, d, c, a, e.

1. b first: nothing can start before the DMAC knows the source address, the destination address and the word count.

2. d next: once the DMAC is programmed, the CPU launches the disk read command and is free to schedule another process.
3. c then: the DMAC, now bus master, moves the block directly into memory by cycle stealing, without involving the CPU.
4. a then: when the word count reaches zero, the DMAC raises the end-of-transfer interrupt to inform the CPU.
5. e last: the CPU runs the ISR, checks the controller's status register for possible errors and takes back full control of the bus.

SOLUTION **Exercise 6 — Completing a comparison table**

Criterion	Polling	Interrupt-driven I/O	DMA
CPU involvement during transfer	Total: CPU tests the status flag and moves every word itself	Partial: CPU moves each word but only when interrupted	Minimal: CPU only programs the DMAC and handles the final interrupt
Relative speed for large blocks	Very slow (busy waiting)	Medium (one interrupt per word or per small unit)	Fast (block moved at bus speed)
Hardware cost / complexity	Lowest: a simple status flag suffices	Medium: interrupt line and controller logic needed	Highest: a DMA controller with bus mastering is required
Two example devices	Simple sensors, washing-machine door switch	Keyboard, mouse	Hard disk, network card

SOLUTION **Exercise 7 — Scenario analysis: the frozen computer**

1. The symptoms point to **programmed I/O (polling)**: the driver is busy-waiting for the faulty device to signal "ready".

2. Step by step: the operating system issues a command to the flash drive; the driver enters a polling loop on the controller's status register; the faulty device never sets the ready flag; the CPU keeps executing the test loop at high priority and therefore cannot schedule the mouse driver or any other task, so the whole machine appears frozen even though the CPU is working at full speed.
3. Possible remedies (any one accepted): a timeout in the driver (after a fixed delay the device is declared failed and the loop is abandoned); using interrupt-driven I/O instead of polling so the CPU remains free while waiting; at hardware level, a watchdog timer that resets a controller which does not respond.

SOLUTION **Exercise 8 — Choosing the right mechanism**

1. Interrupt-driven I/O: key presses are rare and unpredictable; the CPU must stay free for billing and stock tasks between presses, and the cost of one interrupt per key is negligible.
2. DMA: a 4 GB file is a huge block transfer; DMA moves it at bus speed and frees the CPU completely, whereas polling or per-word interrupts would monopolise it.
3. Polling: the microcontroller has nothing else to do, the check is extremely cheap and the response must be immediate and deterministic; a simple test loop is ideal and avoids the complexity of interrupt hardware.
4. DMA (combined with interrupts): packets arrive in large volumes at high speed; per-word interrupts would saturate the CPU, so the network card writes packets straight to memory by DMA and interrupts only once per packet or per buffer.

SOLUTION **Exercise 9 — Structured question: from key press to memory**

Examiner's expectations and mark scheme (10 marks).

1. Device controller (2 marks): the electronic hardware interface circuit between the peripheral and the system bus (1 mark for "hardware"); it contains status, control and data registers and physically operates the device (1 mark). Device driver (2 marks): the software program, part of the operating system (1 mark for "software"); it translates OS requests into commands for the controller and contains the interrupt service routine (1 mark).
2. Diagram and explanation (6 marks): award 2 marks for a clear labelled diagram showing keyboard → controller (with status register and data register) → IRQ line → CPU → memory; then 1 mark for each of the following steps: the key code is placed in the data register and the status flag is set; the controller raises the IRQ line; the CPU sus-

pends its task, saves its context and runs the ISR supplied by the device driver; the ISR reads the data register, stores the character in a memory buffer, and the CPU restores its context and resumes the interrupted program.

Common mistakes: Candidates often swap "controller" and "driver" (the controller is hardware, the driver is software), forget to mention the saving/restoring of the CPU context, or draw the data flowing through the CPU registers without naming the controller's data register.

SOLUTION **Exercise 10 — Essay: a compromise between polling and DMA**

Marking guide (15 marks).

1. Principles (about 5 marks). Polling: the CPU loops on the status flag and moves every word itself (busy waiting). Interrupt-driven I/O: the device raises an IRQ when ready; the CPU runs an ISR that moves the word, then resumes its work. DMA: the CPU programs a DMAC which transfers the whole block directly to memory by bus mastering, interrupting only at the end.

2. Advantages and limitations (about 6 marks: 1 mark per valid point). Polling — advantages: very simple and cheap hardware, predictable response time; limitation: wastes CPU time in busy waiting. Interrupt-driven I/O — advantages: CPU free between events, well suited to slow unpredictable devices; limitation: one interrupt per word is costly for fast devices because of context-saving overhead. DMA — advantages: fastest for large blocks, CPU almost totally free; limitation: requires extra hardware (DMAC) and is overkill for single characters.

3. Conclusion (about 4 marks). A modern personal computer uses all three mechanisms simultaneously: polling inside simple embedded controllers, interrupts for the keyboard and mouse, DMA for the hard disk and the network card. Each mechanism is used where its cost/performance balance is optimal, which confirms that interrupt-driven I/O is indeed the intermediate compromise between the simplicity of polling and the power of DMA.

Examiner's expectations: a structured essay (introduction, development, conclusion), correct technical vocabulary (status register, IRQ, ISR, bus mastering, cycle stealing) and at least three concrete devices correctly matched to mechanisms.

SOLUTION **Exercise 11 — Problem: performance study of a data transfer**

1. (2 marks) Number of bytes: $64 \times 2^{20} = 67\,108\,864$ bytes. Number of 4-byte words:

$$N = \frac{67\,108\,864}{4} = 16\,777\,216 \text{ words} \approx 1.678 \times 10^{7} \text{ words}.$$

2. (4 marks) Polling: 40 instructions per word, hence

$$40 \times 16\,777\,216 = 6.71 \times 10^8 \text{ instructions}, \qquad t = \frac{6.71 \times 10^8}{10^9} \approx 0.671\,\text{s}.$$

3. (4 marks) Interrupt-driven I/O: 40 + 20 = 60 instructions per word, hence

$$60 \times 16\,777\,216 \approx 1.007 \times 10^9 \text{ instructions}, \qquad t \approx 1.007\,\text{s}.$$

   Comment: for a fast block device, interrupt-driven I/O is even slower than polling for the CPU, because the per-interrupt overhead (saving and restoring the context) exceeds the cost of the polling loop; its advantage — a free CPU between events — disappears when the device is always ready.

4. (5 marks) DMA: 5 000 + 2 000 = 7 000 instructions in total, hence

$$t = \frac{7\,000}{10^9} = 7\,\text{≡s}.$$

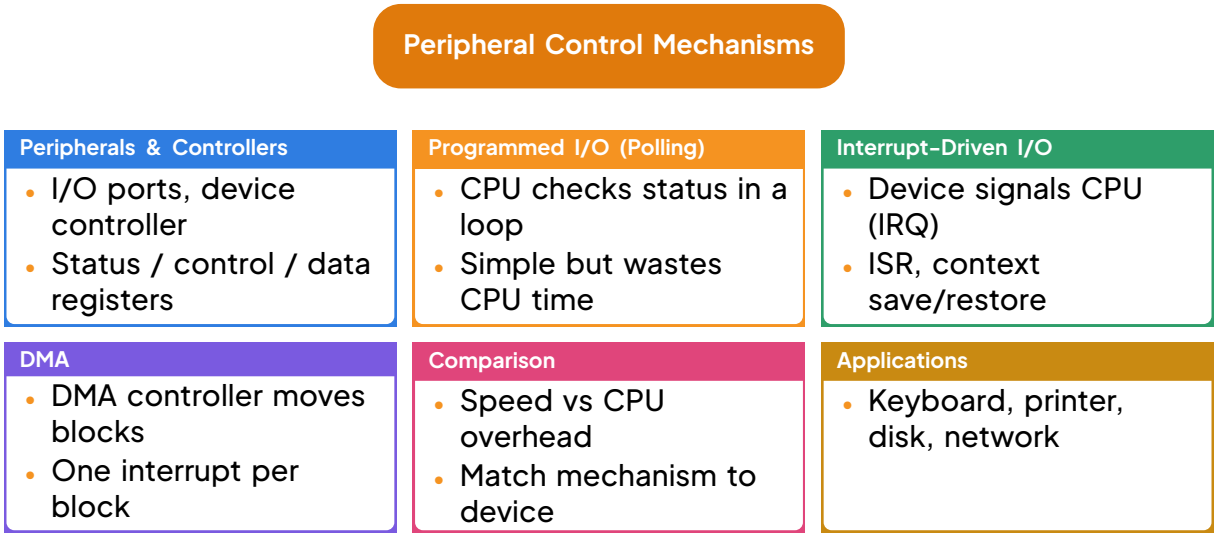
   Percentage of CPU time saved compared with polling:

$$\frac{0.671 - 7 \times 10^{-6}}{0.671} \times 100 \approx 99.999\,\%.$$

   The CPU is essentially free during the whole backup: this is why DMA is the standard mechanism for disk and network transfers.

Examiner's expectations: correct conversion of 64 MB using 2^{20} (not 10^6), explicit formulas before numerical answers, units on every time result, and a relevant comment for part (c).

9 SUMMARY SHEET



Mind map of the chapter: peripheral device control mechanisms.

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key definitions.

- A **peripheral** (I/O device) is any hardware unit connected to the computer to input, output or store data: keyboard, mouse, printer, disk, screen, network card, modem.
- A **device controller** (interface/adapter) is the electronic circuit between the bus and the peripheral; it holds **registers**: data register, status register (busy, ready, error) and control register (commands from the CPU).
- An **I/O port** is the addressable point through which the CPU reads/writes these registers (port-mapped I/O or memory-mapped I/O).
- A **control mechanism** is the strategy used to synchronise data transfer between the fast CPU and slow peripherals.
- An **interrupt** is a hardware signal (IRQ) sent by a device to attract the CPU's attention; the CPU suspends its current work, saves its context, runs the **Interrupt Service Routine** (ISR) located via the **interrupt vector table**, then restores the context and resumes.
- **DMA** (Direct Memory Access): a dedicated **DMA controller** transfers a whole block of data directly between memory and the device, without passing through the CPU, and interrupts the CPU only once at the end.

The three mechanisms to know.

- Programmed I/O (polling): the CPU repeatedly reads the status register in a loop until the device is ready, then transfers the data itself. Simple, no extra hardware, but the CPU wastes time waiting (busy waiting). Suitable for very simple, predictable systems.
- Interrupt-driven I/O: the CPU does other work; the device raises an IRQ when ready. The CPU responds via the ISR. Efficient for slow, unpredictable devices (keyboard, mouse). Overhead: context saving/restoring at every interrupt.
- DMA: the CPU initialises the DMA controller (source/destination address, byte count, direction); the controller transfers the block using cycle stealing on the bus; one interrupt signals completion. Best for fast block devices: hard disk, sound card, network card.

Comparison table (learn it).

Criterion	Polling	Interrupt	DMA
CPU involvement	Total	Per data item	Start and end only
CPU efficiency	Very low	Good	Excellent
Hardware cost	None	Interrupt controller	DMA controller
Best for	Simple devices	Slow devices	Fast block transfers

Methods.

- To analyse a polling sequence: read status register → test ready bit → if not ready, loop back; if ready, transfer data → repeat.
- To describe an interrupt: device raises IRQ → CPU finishes current instruction → saves PC and registers → jumps to ISR via vector table → services device → restores context → resumes.
- To justify a choice of mechanism, always argue from: device speed, data volume, CPU availability and hardware cost.

Common mistakes.

- Confusing the controller (hardware circuit) with the driver (software).
- Saying DMA needs "no CPU at all": the CPU must still configure the transfer and is interrupted at the end.
- Believing interrupts make polling disappear: the ISR itself may use programmed I/O to move the data.
- Forgetting that polling wastes CPU time even when the device is not ready (busy waiting).
- Mixing up the roles of the status register (device → CPU) and the control register (CPU → device).

GCE tips.

- Typical questions: “Define interrupt”, “Distinguish polling from interrupt-driven I/O”, “Explain the steps of a DMA transfer”, “Give one advantage and one disadvantage of each mechanism”.
- Always support a comparison with a clear table or a step-numbered sequence; examiners reward structure.
- Use concrete examples: keyboard (interrupt), printer (interrupt/polling), hard disk and network card (DMA) — e.g. a modem handling a mobile-money transaction at an MTN or Orange kiosk relies on interrupt-driven I/O.
- Define every keyword you use (IRQ, ISR, vector, cycle stealing) before using it in an argument.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+